

Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai
Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.



Department of Information Technology

CERTIFICATE

This is to certify that **Aditi M.Taksale** of **D15A** semester **VI**, have successfully completed necessary experiments in the **MAD & PWA Lab** under my supervision in **VES Institute of Technology** during the academic year **2024-2025**.

Lab Assistant

Subject Teacher

Mrs. Kajal Joseph

Principal

Head of Department

Dr. Mrs. Shalu Chopra

Name of the Course : MAD & PWA Lab

Course Code : ITL604

Year/Sem/Class : D15A

A.Y.: 24-25

Faculty Incharge : Mrs. Kajal Joseph.

Lab Teachers : Mrs. Kajal Joseph.

Email : kajal.jewani@ves.ac.in

Programme Outcomes: The graduate will be able to:

PO1) Basic Engineering knowledge: An ability to apply the fundamental knowledge in mathematics, science and engineering to solve problems in Computer engineering.

PO2) Problem Analysis: Identify, formulate, research literature and analyze computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and computer engineering and sciences.

PO3) Design/ Development of Solutions: Design solutions for complex computer engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4) Conduct investigations of complex engineering problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5) Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern computer engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6) The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to computer engineering practice.

PO7) Environment and Sustainability: Understand the impact of professional computer engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of computer engineering practice.

PO9) Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11) Project Management and Finance: Demonstrate knowledge and understanding of computer engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12) Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Program specific Outcomes

PSO1) An ability to manage and analyze data / information effectively for making better decisions.

PSO2) Demonstrate the ability to use state of the art technologies and tools including Free and Open Source Software (FOSS) tools in developing software.

Lab Objectives:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	Learn the basics of the Flutter framework.
2	Develop the App UI by incorporating widgets, layouts, gestures and animation
3	Create a production ready Flutter App by including files and firebase backend service.
4	Learn the Essential technologies, and Concepts of PWAs to get started as quickly and efficiently as possible
5	Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6	Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

Sr. No.	Lab Outcomes	Cognitive levels of attainment as per Bloom's Taxonomy
On Completion of the course the learner/student should be able to:		
1	Understand cross platform mobile application development using Flutter framework	L1, L2
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	L3
3	Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS	L3, L4
4	Understand various PWA frameworks and their requirements	L1, L2
5	Design and Develop a responsive User Interface by applying PWA Design techniques	L3
6	Develop and Analyse PWA Features and deploy it over app hosting solutions	L3, L4

Index

Sr. No	Experiment Title	LO	Grade
1.	To install and configure the Flutter Environment	LO1	
2.	To design Flutter UI by including common widgets.	LO2	
3.	To include icons, images, fonts in Flutter app	LO2	
4.	To create an interactive Form using form widget	LO2	
5.	To apply navigation, routing and gestures in Flutter App	LO2	
6.	To Connect Flutter UI with fireBase database	LO3	
7.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.	LO4	
8.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA	LO5	
9.	To implement Service worker events like fetch, sync and push for E-commerce PWA	LO5	
10.	To study and implement deployment of Ecommerce PWA to GitHub Pages.	LO5	
11.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.	LO6	

MAD & PWA Lab

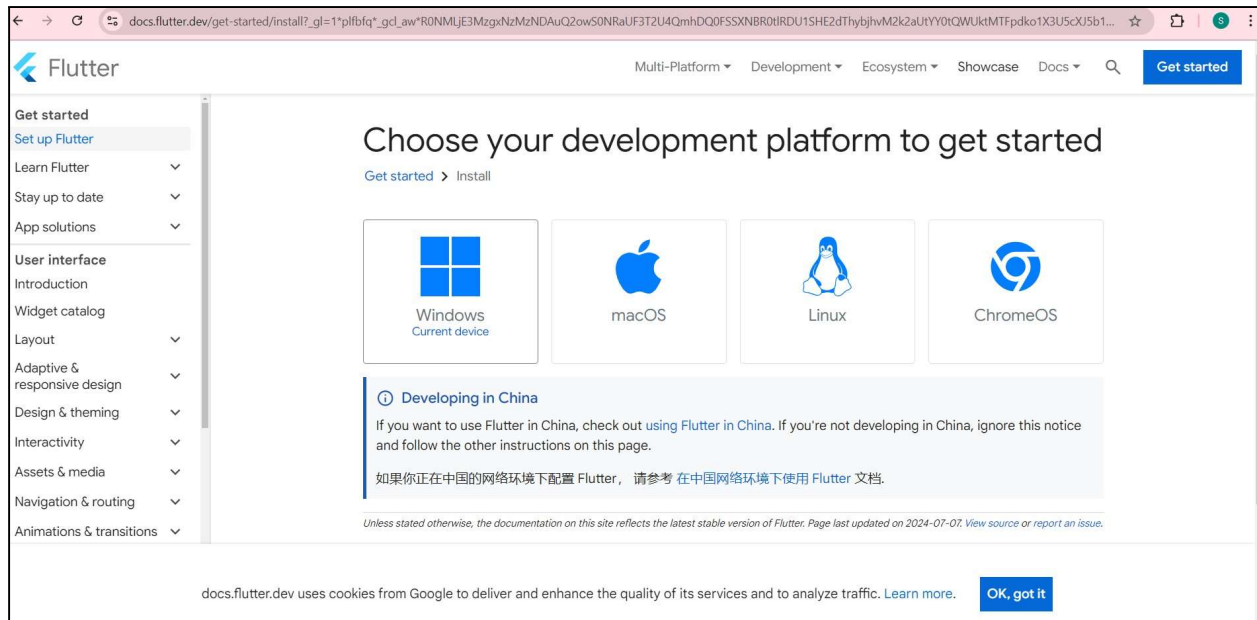
Journal

Experiment No.	01
Experiment Title.	To install and configure the Flutter Environment
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework
Grade:	

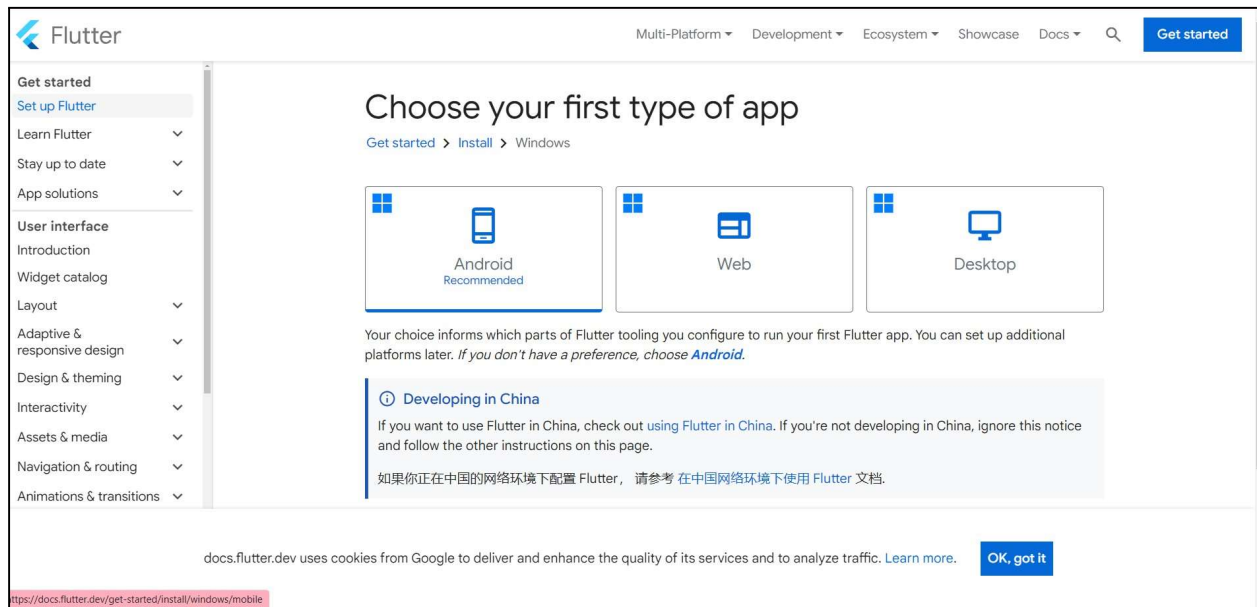
Experiment No. 1

AIM : Installation and Configuration of Flutter Environment.

Step 1: Download the installation bundle of the Flutter Software Development Kit for windows. To download Flutter SDK, Go to its official website <https://docs.flutter.dev/get-started/install> , you will get the following screen.



Step 2: Next, to download the latest Flutter SDK, click on the Windows icon and then select Android. Here, you will find system requirements and the download link for SDK.



Flutter

Multi-PlatformDevelopmentEcosystemShowcaseDocs

Get started

Get started

Set up Flutter

Learn Flutter

Stay up to date

App solutions

User interface

Introduction

Widget catalog

Layout

Adaptive & responsive design

Design & theming

To install the Flutter SDK, you can use the VS Code Flutter extension or download and install the Flutter bundle yourself.

Use VS Code to installDownload and install

Download then install Flutter #

To install Flutter, download the Flutter SDK bundle from its archive, move the bundle to where you want it stored, then extract the SDK.

1. Download the following installation bundle to get the latest stable release of the Flutter SDK.

flutter_windows_3.27.3-stable.zip

For other release channels, and older builds, check out the SDK archive.

The Flutter SDK should download to the Windows default download directory:

%USERPROFILE%\Downloads

Contents

Verify system requirements

Hardware requirements

Software requirements

Configure a text editor or IDE

Install the Flutter SDK

Configure Android development

Configure the Android toolchain in Android Studio

Configure your target Android device

docs.flutter.dev uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic. Learn more.

OK, got it

72°FSmoke

Search

ENG

9:21 AM1/28/2025

Flutter

Multi-PlatformDevelopmentEcosystem

Get started

Get started

Set up Flutter

Learn Flutter

Stay up to date

App solutions

User interface

Introduction

Widget catalog

Layout

Adaptive & responsive design

Design & theming

To install the Flutter SDK, you can use the VS Code Flutter extension or download and install the yourself.

Use VS Code to installDownload and install

Download then install Flutter

To install Flutter, download the Flutter SDK bundle from its archive, move the bundle to where you want it stored, then extract the SDK.

1. Download the following installation bundle to get the latest stable release of the Flutter SDK.

flutter_windows_3.27.3-stable.zip

For other release channels, and older builds, check out the SDK archive.

The Flutter SDK should download to the Windows default download directory:

%USERPROFILE%\Downloads

Recent download history

flutter_windows_3.27.3-stable.zip

4 0.1/1.0 GB • 11 minutes left

Full download history

Verify system requirements

Hardware requirements

Software requirements

Configure a text editor or IDE

Install the Flutter SDK

Configure Android development

Configure the Android toolchain in Android Studio

Configure your target Android device

docs.flutter.dev uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic. Learn more.

OK, got it

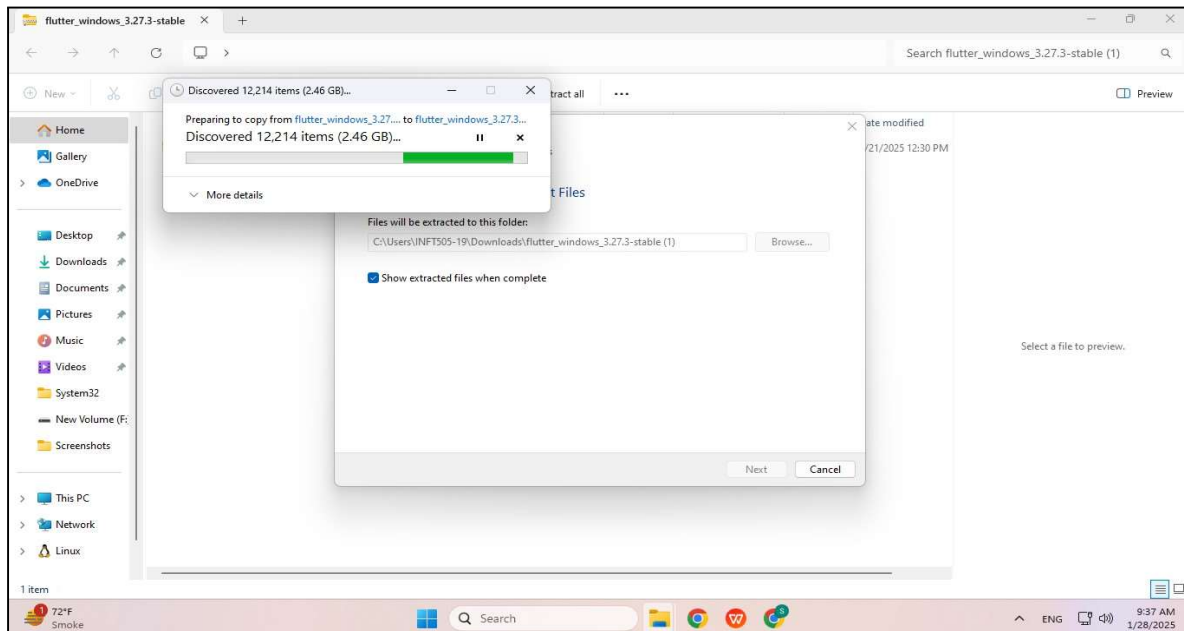
72°FSmoke

Search

ENG

9:22 AM1/28/2025

Step 3: When your download is complete, extract the zip file and place it in the desired installation folder or location, for example, C: /Flutter. (Here I have created Flutter folder in C drive and inside that created src folder and extracted in it.)



Step 4: Now run flutter doctor command in that folder bin directory we will get to know the status and what is remaining to install

```
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\flutter_windows_3.27.3-stable\flutter\bin>flutter doctor

Welcome to Flutter! - https://flutter.dev

The Flutter tool uses Google Analytics to anonymously report feature usage
statistics and basic crash reports. This data is used to help improve
Flutter tools over time.

Flutter tool analytics are not sent on the very first run. To disable
reporting, type 'flutter config --no-analytics'. To display the current
setting, type 'flutter config'. If you opt out of analytics, an opt-out
event will be sent, and then no further information will be sent by the
Flutter tool.

By downloading the Flutter SDK, you agree to the Google Terms of Service.
The Google Privacy Policy describes how data is handled in this service.

Moreover, Flutter includes the Dart SDK, which may send usage metrics and
crash reports to Google.

Read about data we send with crash reports:
https://flutter.dev/to/crash-reporting

See Google's privacy policy:
https://policies.google.com/privacy

To disable animations in this tool, use
'flutter config --no-cli-animations'.
```

```

Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.3, on Microsoft Windows [Version 10.0.22631.4751], locale en-IN)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[X] Android toolchain - develop for Android devices
    X Unable to locate Android SDK.
      Install Android Studio from: https://developer.android.com/studio/index.html
      On first launch it will assist you in installing the Android SDK components.
      (or visit https://flutter.dev/to/windows-android-setup for detailed instructions).
      If the Android SDK has been installed to a custom location, please use
      'flutter config --android-sdk' to update to that location.

[✓] Chrome - develop for the web
[X] Visual Studio - develop Windows apps
    X Visual Studio not installed; this is necessary to develop Windows apps.
      Download at https://visualstudio.microsoft.com/downloads/.
      Please install the "Desktop development with C++" workload, including all of its default components
[!] Android Studio (not installed)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources

! Doctor found issues in 3 categories.

The Flutter CLI developer tool uses Google Analytics to report usage and diagnostic
data along with package dependencies, and crash reporting to send basic crash
reports. This data is used to help improve the Dart platform, Flutter framework,
and related tools.

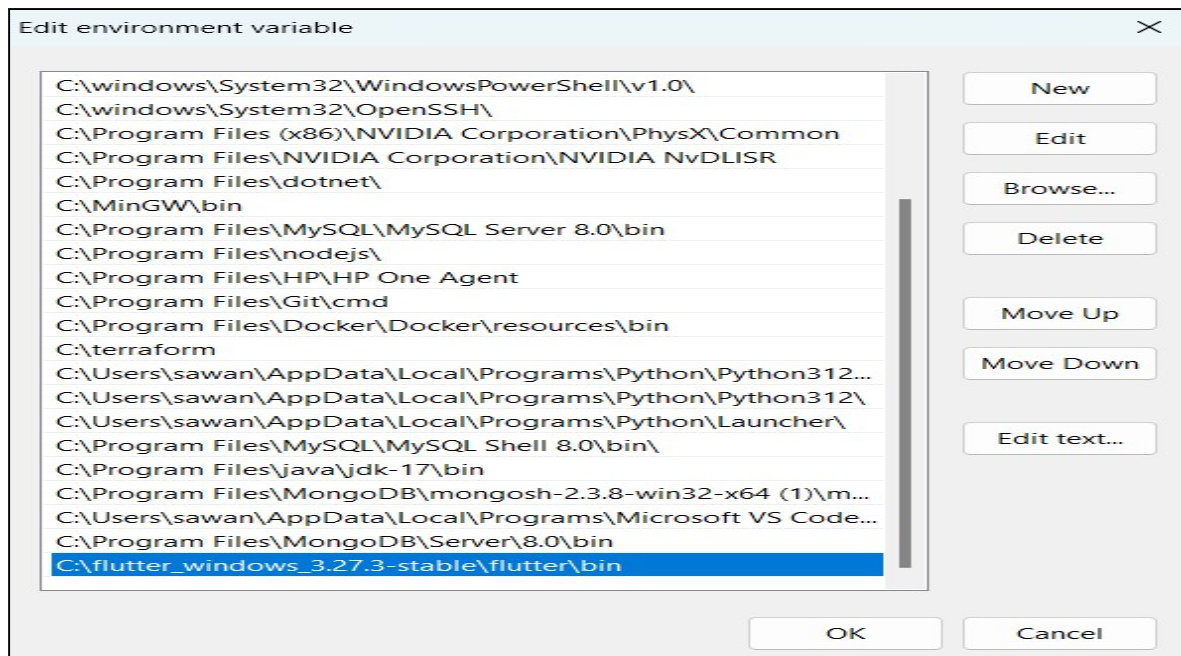
Telemetry is not sent on the very first run. To disable reporting of telemetry,
run this terminal command:

    flutter --disable-analytics

If you opt out of telemetry, an opt-out event will be sent, and then no further
information will be sent. This data is collected in accordance with the Google
Privacy Policy (https://policies.google.com/privacy).

```

Step 5: To run the Flutter command in regular windows console, you need to update the system path to include the flutter bin directory. The following steps are required to do this: Step 5.1: Search for environment variables in search bar -> advanced tab -> environment variables. You will get the following screen.



Step 6: Now, run the \$ flutter command in command prompt.

```
PS C:\Users\sawan> flutter
Manage your Flutter app development.

Common commands:

  flutter create <output directory>
    Create a new Flutter project in the specified directory.

  flutter run [options]
    Run your Flutter application on an attached device or in an emulator.

Usage: flutter <command> [arguments]

Global options:
-h, --help                Print this usage information.
-v, --verbose              Noisy logging, including all shell commands executed.
                           If used with "--help", shows hidden options. If used with "flutter doctor", shows additional diagnostic information. (Use "--vv"
                           to force verbose logging in those cases.)
-d, --device-id            Target device id or name (prefixes allowed).
--version                 Reports the version of this tool.
--enable-analytics         Enable telemetry reporting each time a flutter or dart command runs.
--disable-analytics       Disable telemetry reporting each time a flutter or dart command runs, until it is re-enabled.
--suppress-analytics       Suppress analytics reporting for the current CLI invocation.

Available commands:

Flutter SDK
bash-completion  Output command line shell completion setup scripts.
channel          List or switch Flutter channels.
config           Configure Flutter settings.
doctor           Show information about the installed tooling.
downgrade        Downgrade Flutter to the last active version for the current channel.
precache         Populate the Flutter tool's cache of binary artifacts.
upgrade          Upgrade your copy of Flutter.

Project
analyze          Analyze the project's Dart code.
assemble        Assemble and build Flutter resources.
build            Build an executable app or install bundle.
```

Step 7: Now, run the \$ flutter doctor command. This command checks for all the requirements of Flutter app development and displays a report of the status of your Flutter installation.

Step 8: When you run the above command, it will analyze the system and show its report, as shown in the below image. Here, you will find the details of all missing tools, which are required to run Flutter as well as the development tools that are available but not connected with the device.

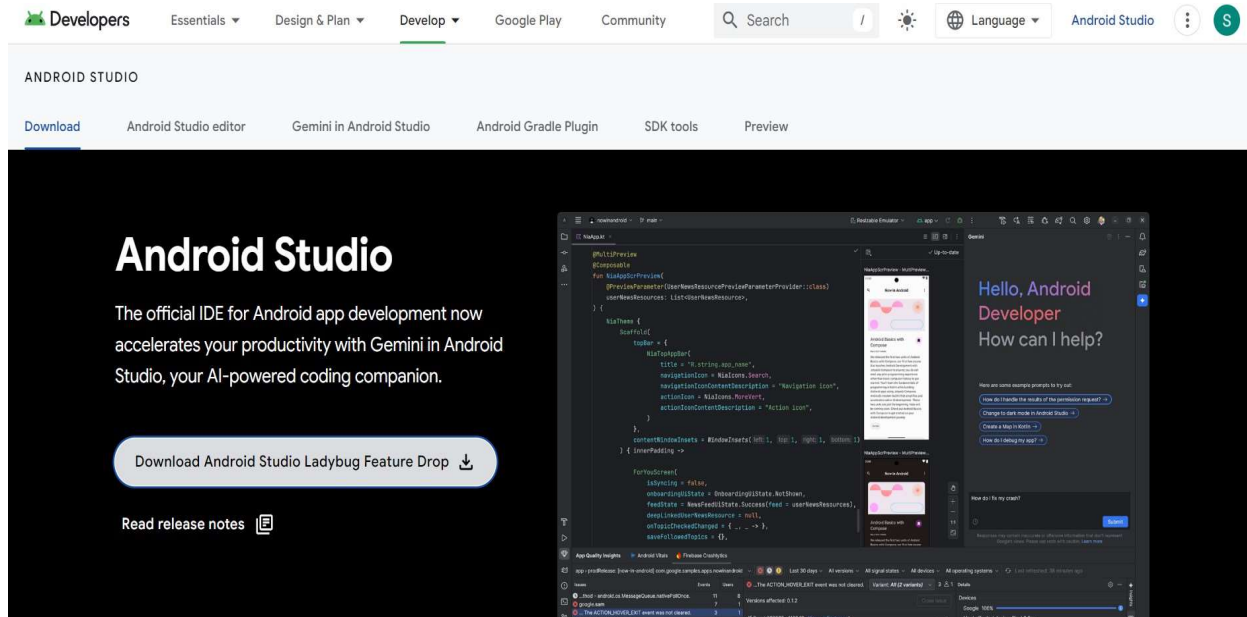
```
PS C:\Users\sawan> flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.3, on Microsoft Windows [Version 10.0.22631.4751], locale en-IN)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✗] Android toolchain - develop for Android devices
    ✗ Unable to locate Android SDK.
       Install Android Studio from: https://developer.android.com/studio/index.html
       On first launch it will assist you in installing the Android SDK components.
       (or visit https://flutter.dev/to/windows-android-setup for detailed instructions).
       If the Android SDK has been installed to a custom location, please use
       'flutter config --android-sdk' to update to that location.

[✓] Chrome - develop for the web
[✗] Visual Studio - develop Windows apps
    ✗ Visual Studio not installed; this is necessary to develop Windows apps.
       Download at https://visualstudio.microsoft.com/downloads/.
       Please install the "Desktop development with C++" workload, including all of its default components
[!] Android Studio (not installed)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[!] Network resources
    ✗ A cryptographic error occurred while checking "https://maven.google.com/": Connection terminated during handshake
       You may be experiencing a man-in-the-middle attack, your network may be compromised, or you may have malware installed on your computer.
    ✗ A cryptographic error occurred while checking "https://github.com/": Connection terminated during handshake
       You may be experiencing a man-in-the-middle attack, your network may be compromised, or you may have malware installed on your computer.

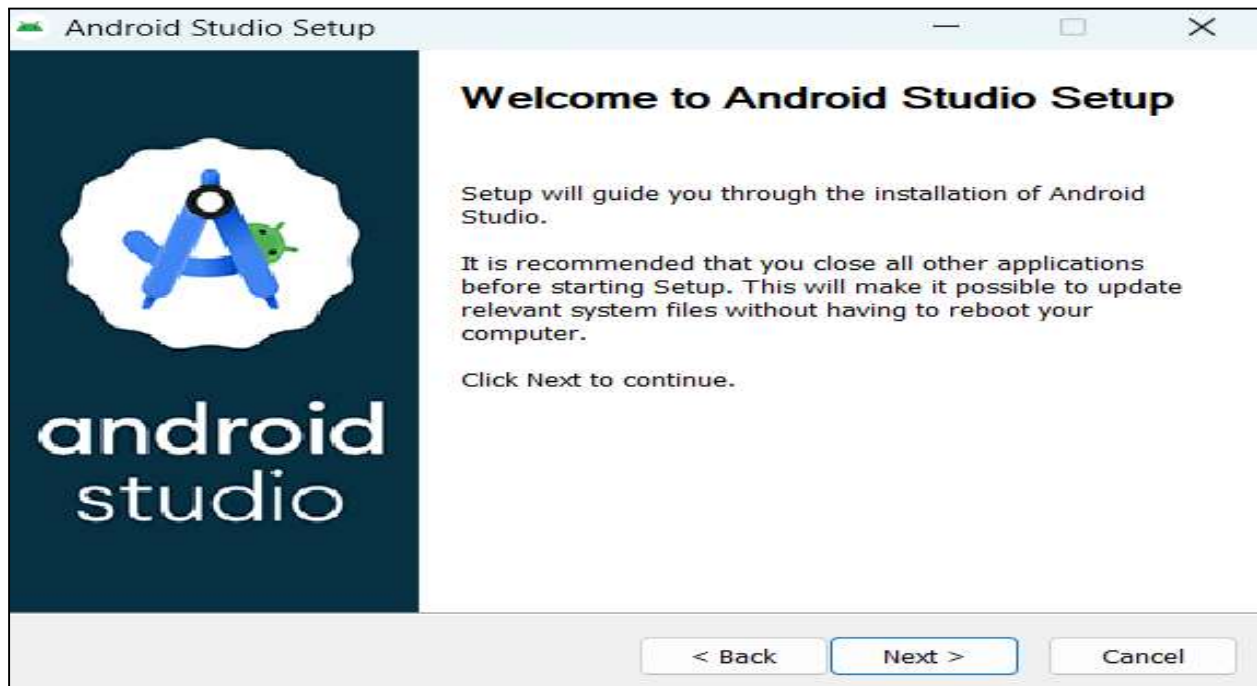
! Doctor found issues in 4 categories.
```

Step 9: Install the Android SDK. If the flutter doctor command does not find the Android SDK tool in your system, then you need first to install the Android Studio IDE. To install Android Studio IDE, do the following steps.

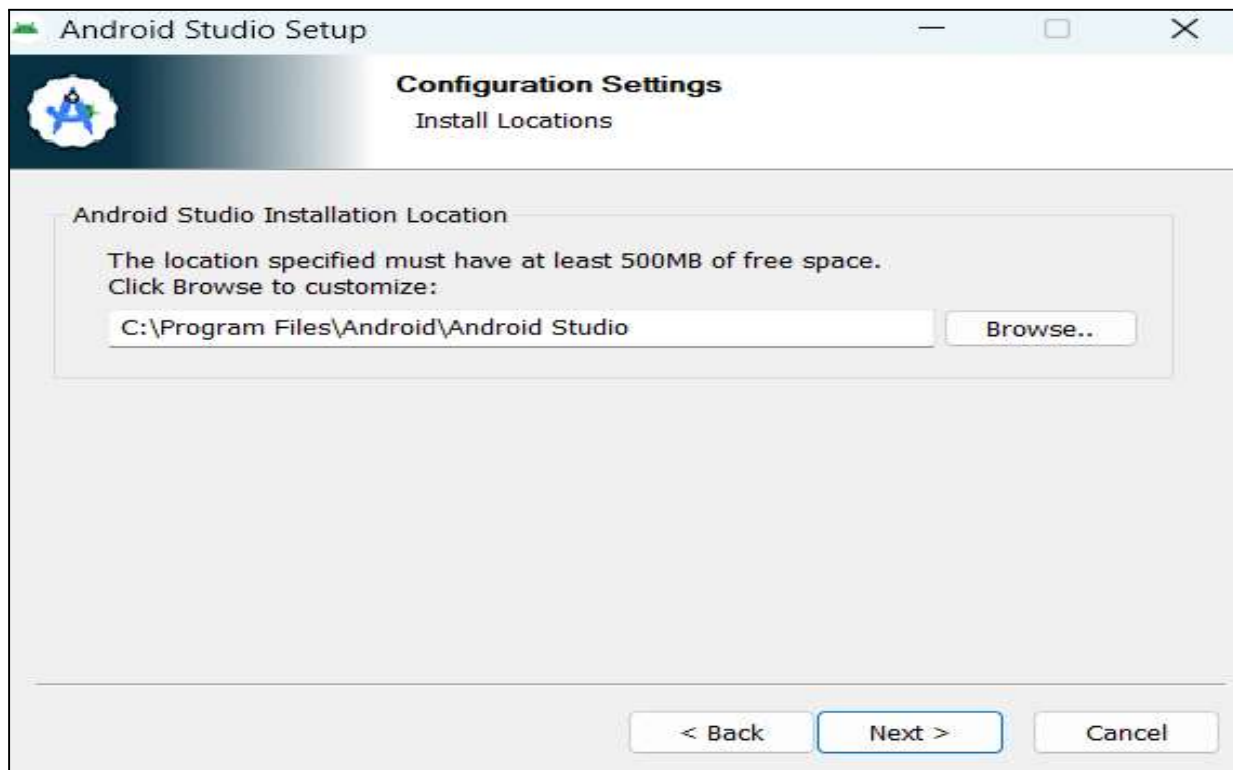
Step 9.1: Download the latest Android Studio executable or zip file from the official site by accepting terms and conditions.



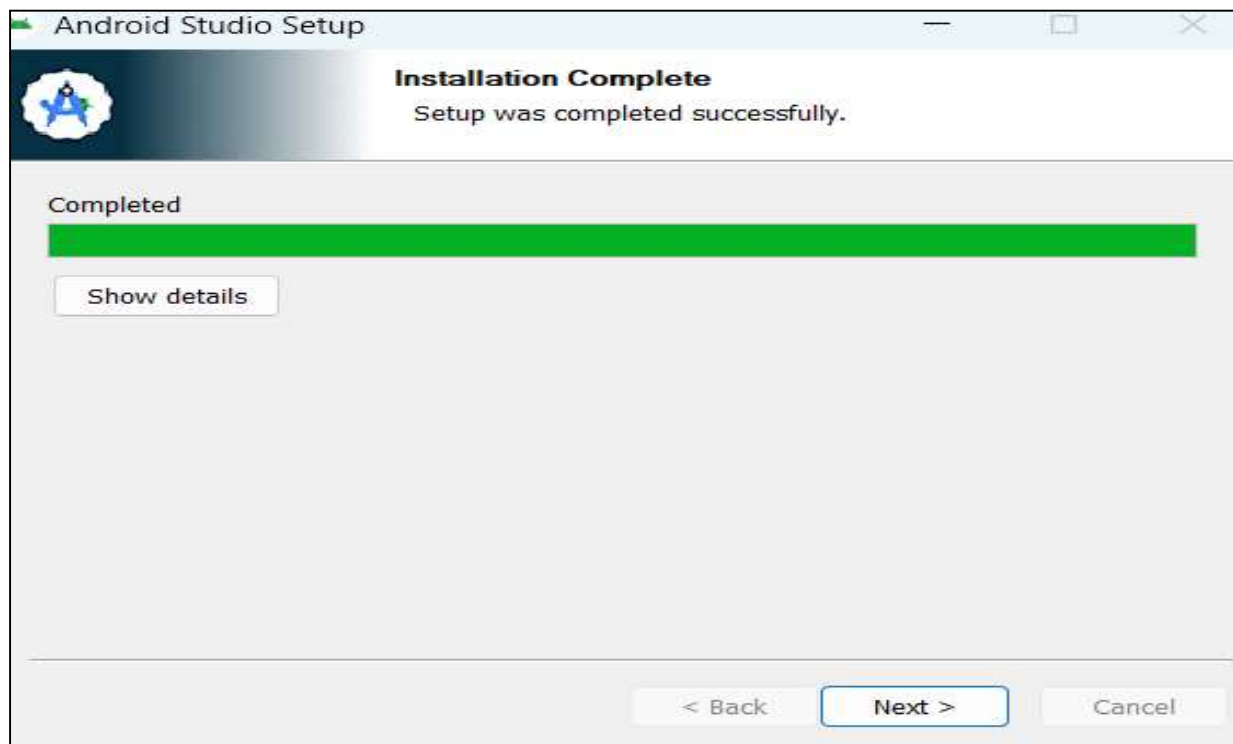
Step 9.2: When the download is complete, open the .exe file and run it. You will get the following dialog box.



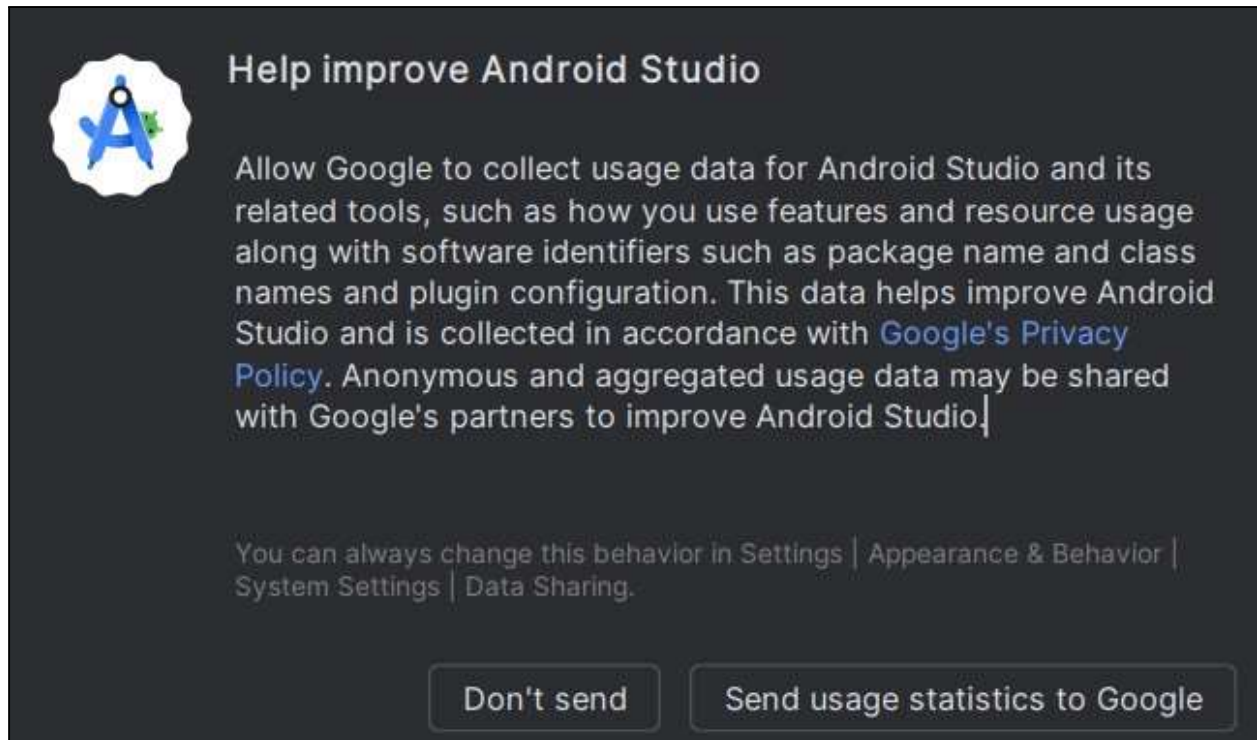
Step 9.3: Follow the steps of the installation wizard and also select installation location.



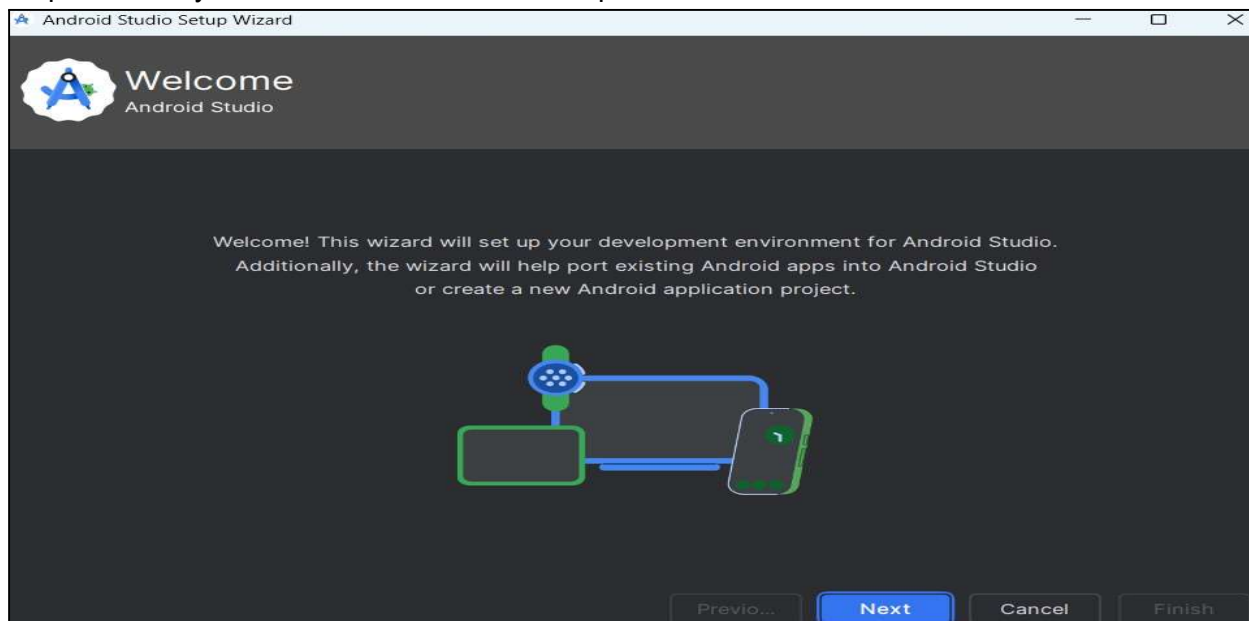
Step 9.4: Once the installation wizard completes, you will get the following screen.



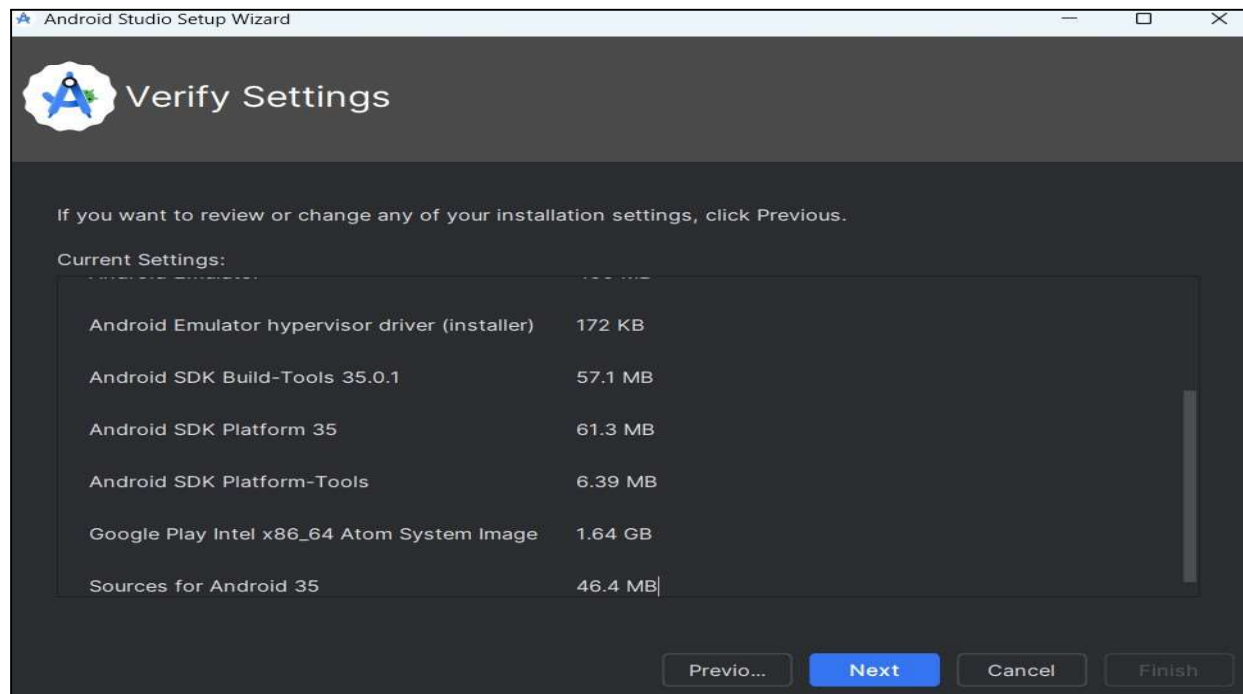
Step 9.5: In the above screen, click Next-> Finish. Once the Finish button is clicked, you need to choose the 'Don't import Settings option' and click OK. It will start the Android Studio. Also click on Don't send so that your data will not get shared with android studio.



Step 9.6: Now you will see android studio setup wizard. Click next



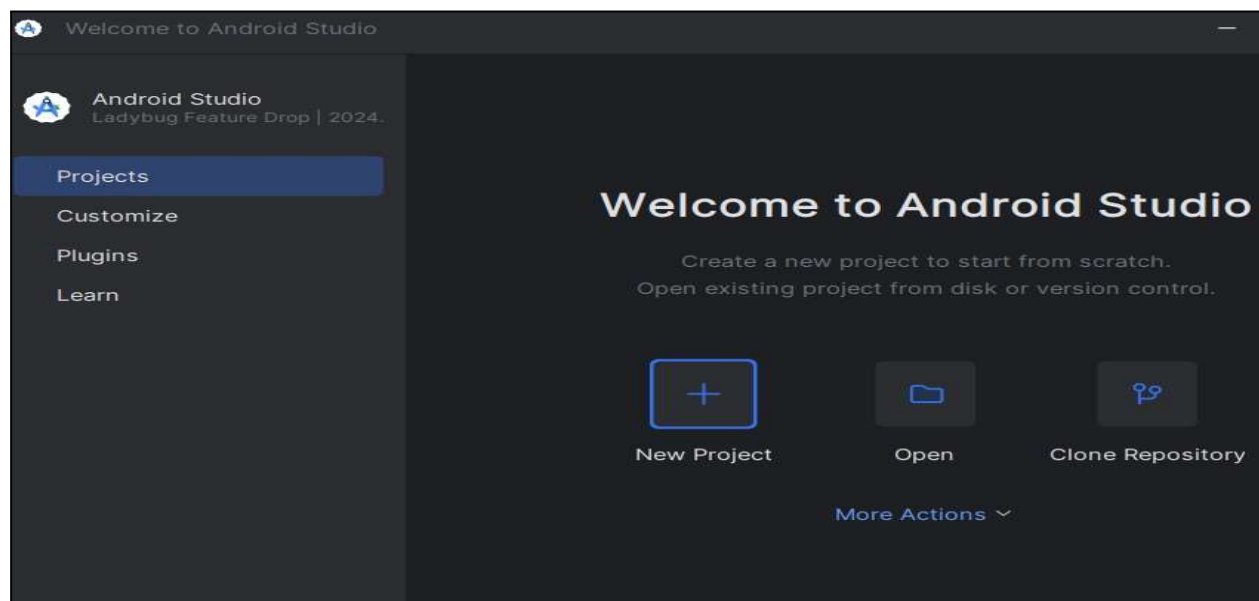
Step 9.7: Select Standard then Next -> Next -> Accept -> Finish.

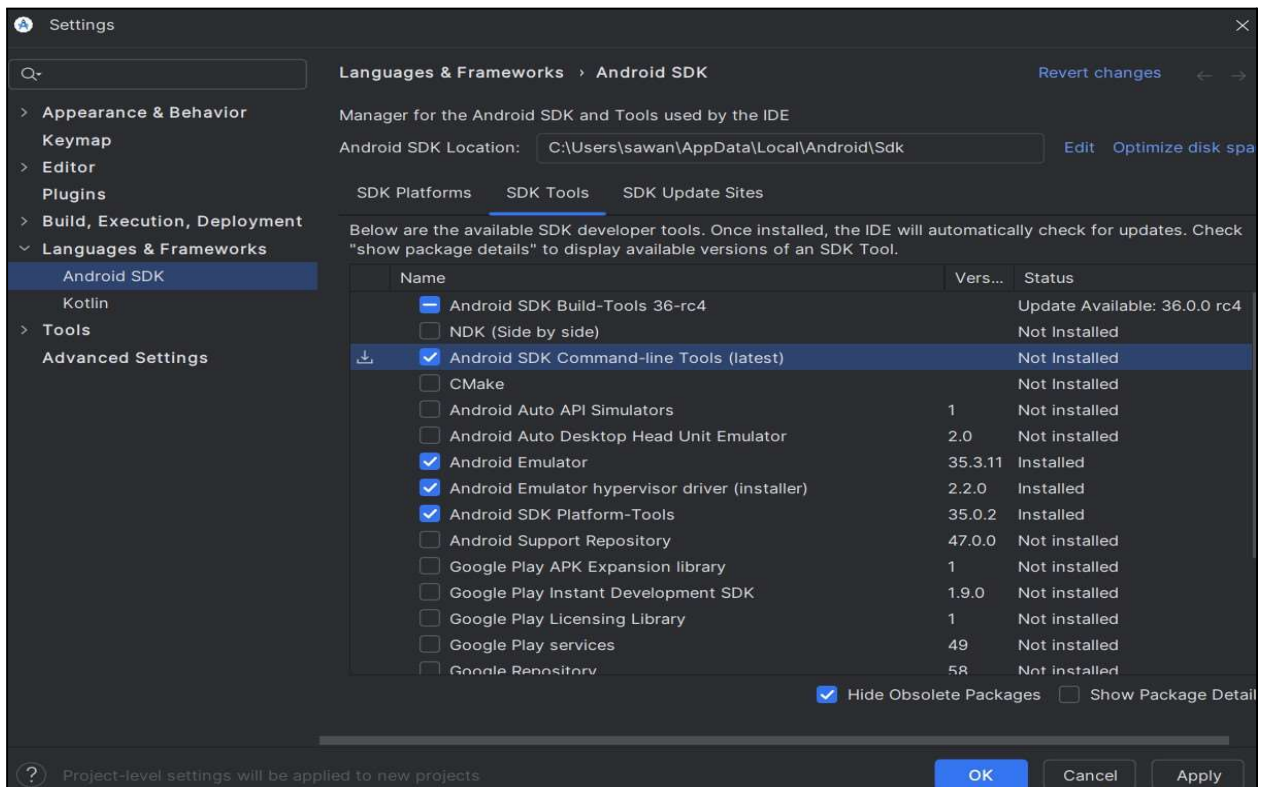
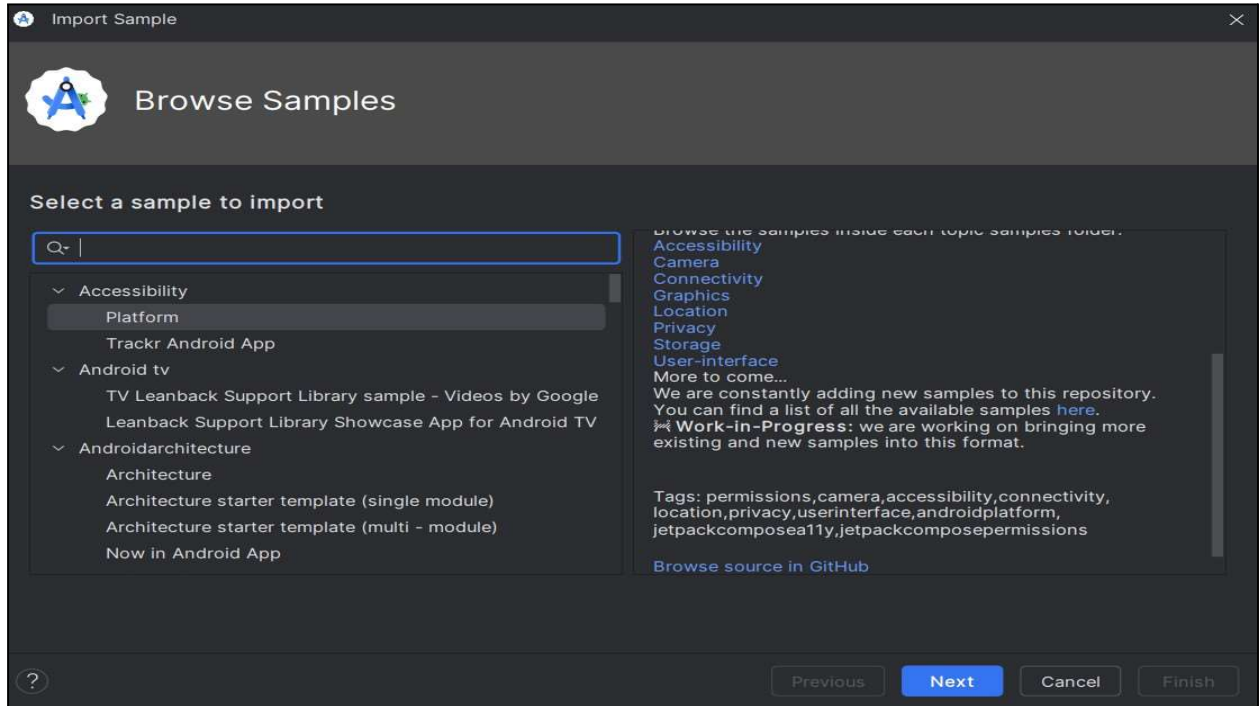


Step 9.8: Now you will see following downloading components wizard. After finishing download click on Finish.

Step 9.9: run the \$ flutter doctor command and Run flutter doctor --android-licenses command.

Step 9.10: Now open android studio you will see the following window. Click on more actions-> Import an android code Sample -> select Android SDK command-line tools (latest) this will download command-line tools.







SDK Component Installer

Completing Requested Actions

SDK Path: C:\Users\sawan\AppData\Local\Android\Sdk

Packages to install: - Android SDK Command-line Tools (latest) (cmdline-tools;latest)

Preparing "Install Android SDK Command-line Tools (latest) v.17.0".

Downloading https://dl.google.com/android/repository/commandlinetools-win-12700392_latest.zip

Starting download...

https://dl.google.com/android/repository/commandlinetools-win-12700392_latest.zip

Please wait until the requested actions are completed.

Background

Cancel

Finish

Step 10: Next, you need to set up an Android emulator. It is responsible for running and testing the Flutter application.

Step 10.1: To set an Android emulator, go to Android Studio > Tools > Android > AVD Manager and select Create Virtual Device. Or, go to Help->Find Action->Type Emulator in the search box. You will get the following screen.

Step 10.2: Choose your device definition and click on Next.

Step 10.3: Select the system image for the latest Android version and click on Next.

```
C:\Users\sawan> flutter doctor --android-licenses
Warning: Errors during XML parse:
Warning: Additionally, the fallback loader failed to parse the XML.ry...
Warning: Errors during XML parse:
Warning: Additionally, the fallback loader failed to parse the XML.ry...
[=====] 100% Computing updates...
6 of 7 SDK package licenses not accepted.
Review licenses that have not been accepted (y/N)? y

1/6: License android-googletv-license:
-----
Terms and Conditions

This is the Google TV Add-on for the Android Software Development Kit License Agreement.

1. Introduction

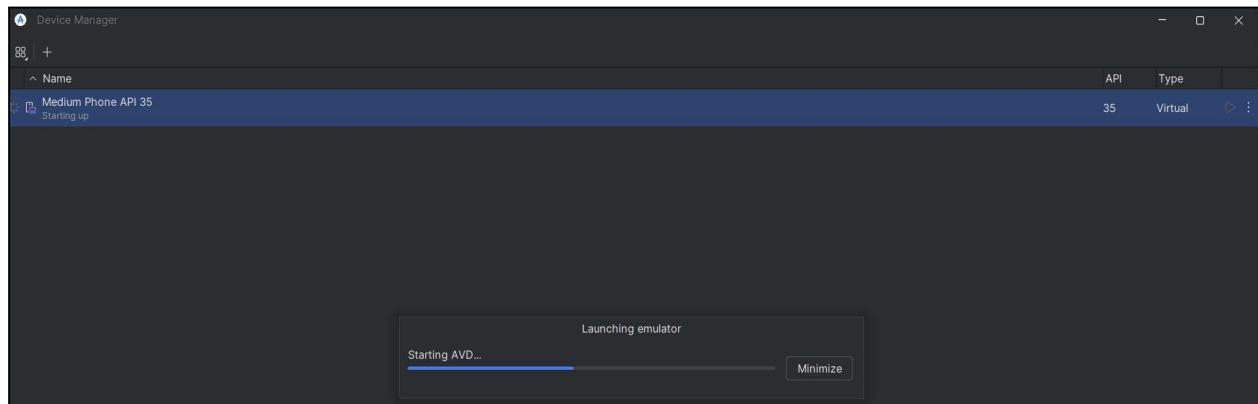
1.1 The Google TV Add-on for the Android Software Development Kit (referred to in this License Agreement as the "Google TV Add-on" and specifically including the Android system files, packaged APIs, and Google APIs add-ons) is licensed to you subject to the terms of this License Agreement. This License Agreement forms a legally binding contract between you and Google in relation to your use of the Google TV Add-on.

1.2 "Google" means Google Inc., a Delaware corporation with principal place of business at 1600 Amphitheatre Parkway, Mountain View, CA 94043, United States.

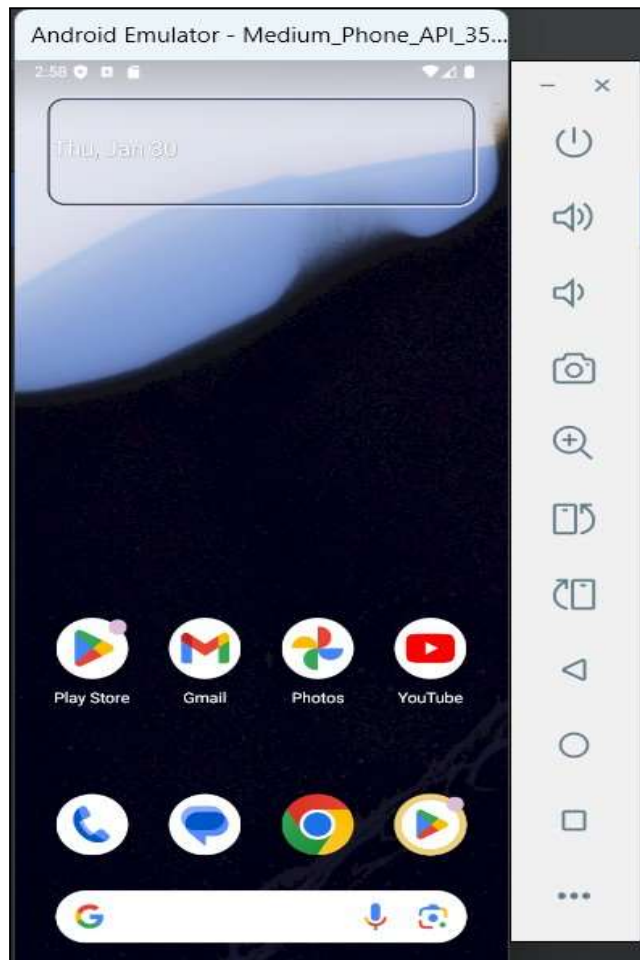
2. Accepting this License Agreement

2.1 In order to use the Google TV Add-on, you must first agree to this License Agreement. You may not use the Google TV Add-on if you do not accept this License Agreement.
```

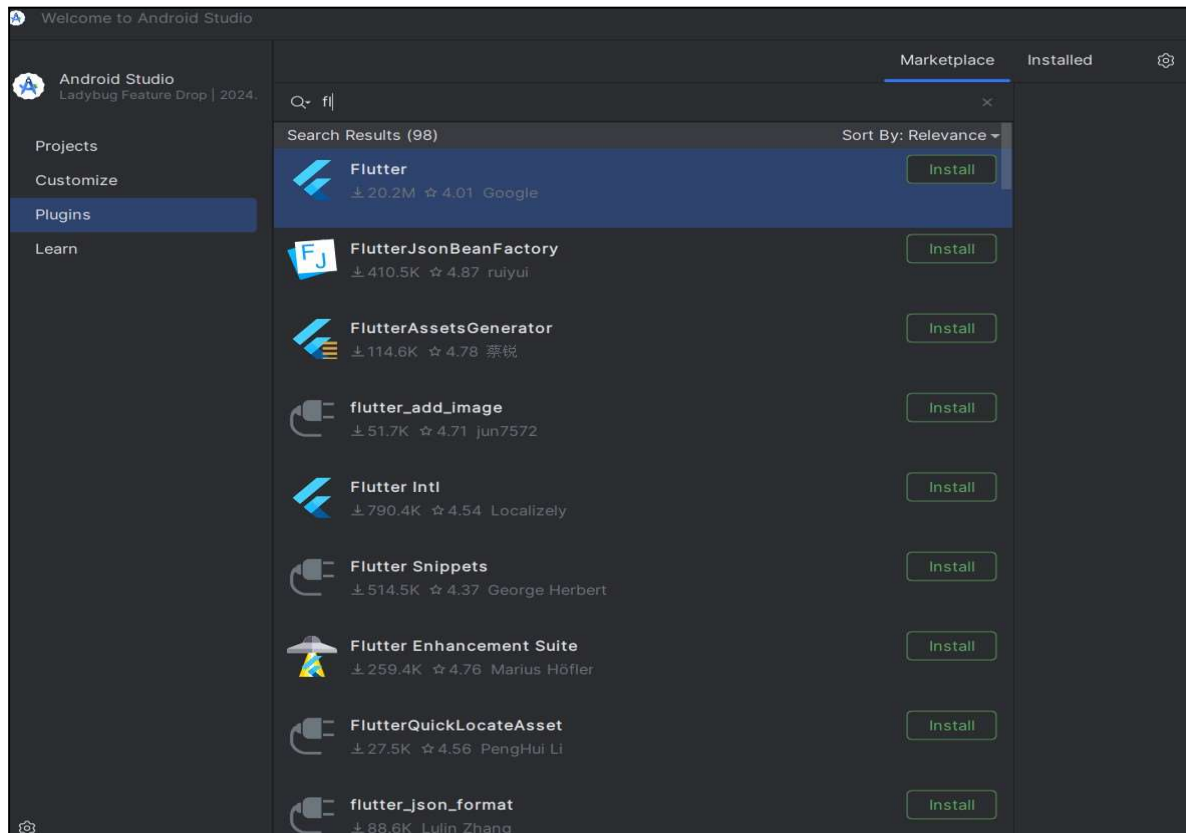
Step 10.4: Now, verify the all AVD configuration. Select Hardware in graphics. If it is correct, click on Finish



Step 10.5: Last, click on the icon pointed into the rectangle. The Android emulator displayed as below screen.

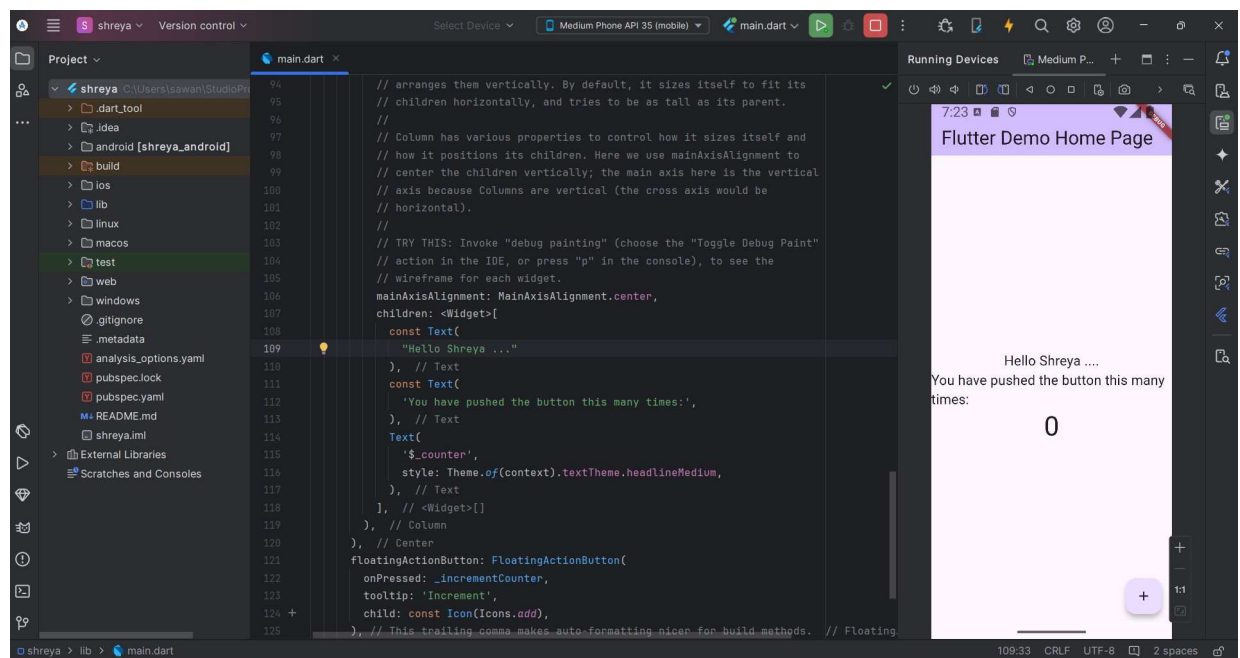
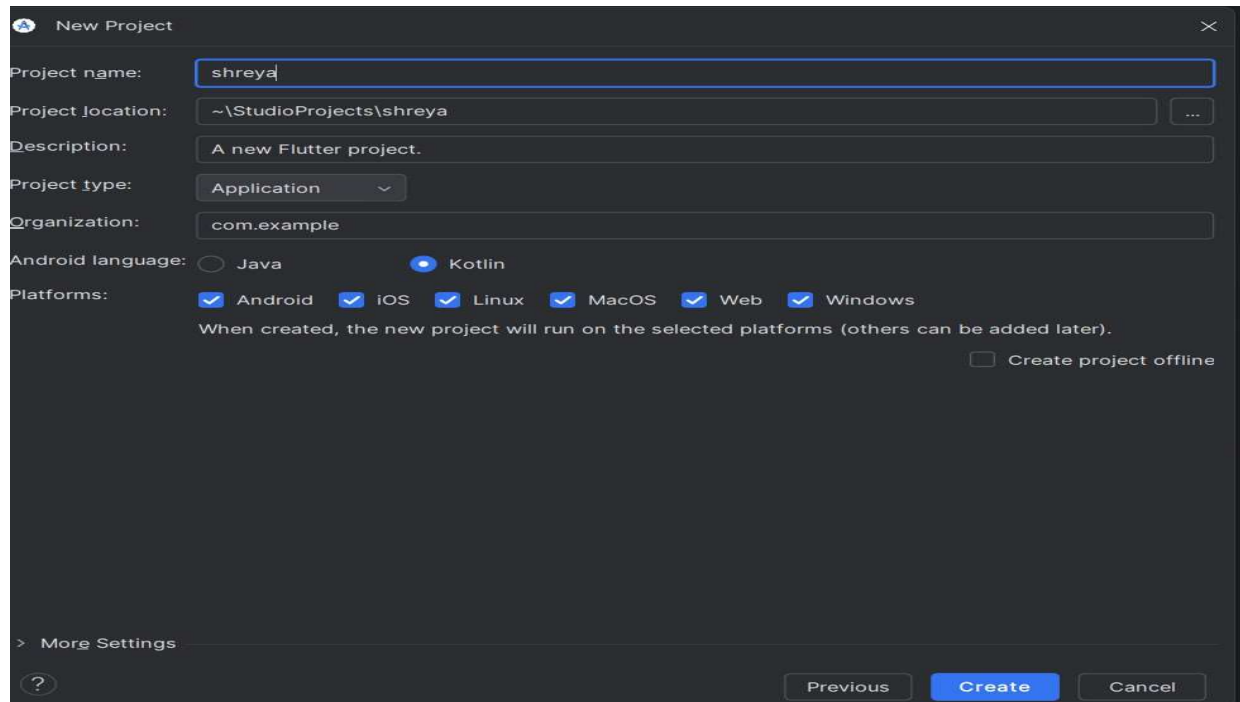


Step 11: Next, you need to install flutter plugin . Go to plugins ->Install Flutter plugin(it will automatically install dart) and then restart android studio.These plugins provide a template to create a Flutter application, give an option to run and debug Flutter application in the Android Studio itself



Step 12: Next, click on new Flutter project and provide SDK path ->Next.

Step 12.1: Next, provide name of the project -> select project location -> create.



MAD & PWA Lab

Journal

Experiment No.	02
Experiment Title.	To design Flutter UI by including common widgets.
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment No. 2

Aim:

To design Flutter UI by including common widgets.

Theory:

In Flutter, designing UIs involves combining various widgets to build interactive and visually appealing applications. Here's a detailed overview of key concepts:

1. Widgets in Flutter

Everything in Flutter is a widget. Widgets are the building blocks of the UI. There are two main types:

- **Stateless Widgets:** These are immutable and don't change over time. They are responsible for displaying UI based on fixed data or input.
- **Stateful Widgets:** These can change their state over time. They are dynamic and are used when the UI needs to update in response to user interaction or other factors.

2. Layout Widgets

The layout of your UI is primarily constructed using widgets like:

- **Container:** A versatile widget used to hold other widgets and apply styling such as padding, margin, colors, and shapes.
- **Column:** A widget that arranges its children vertically. It's useful for stacking widgets in a vertical list.
- **Row:** A widget that arranges its children horizontally. It's useful for placing widgets side by side.
- **Expanded:** A widget that can be used inside a Column, Row, or Flex to make child widgets flexible and fill available space.

3. Text and Icons

- **Text:** The Text widget is used to display static or dynamic text. It can be styled with custom fonts, sizes, colors, and more.
- **Icon:** Flutter provides a large set of material design icons, and the Icon widget lets you display them in various sizes and colors.

4. Buttons and User Interactions

- Flutter provides multiple button widgets like **ElevatedButton**, **TextButton**, and **IconButton** to handle user interaction. These widgets can trigger actions when tapped.
- **TextField**: Used for user input. You can configure it to accept different types of text, such as email or password.
- **Checkbox, Radio, and Switch**: Used for boolean selections, allowing users to choose options in forms or settings.

5. Navigation

- Flutter's **Navigator widget** is responsible for managing routes or screens. You use **Navigator.push** to navigate to a new screen and **Navigator.pop** to return to the previous one.
- Routes define the pages of an app, and you can pass data between them using arguments.

6. Displaying Lists and Grids

- **ListView**: The ListView widget is used to display a list of items that can scroll. It's perfect for long lists that need to be dynamically generated.
- **GridView**: This widget allows you to display items in a grid format, with configurable row and column layouts.

7. State Management

- Flutter provides a variety of ways to manage state. The simplest approach is using **setState()** to update the UI.
- For more complex apps, you can use state management solutions like **Provider**, **Riverpod**, or **Bloc** to separate business logic from UI code.
- Proper state management ensures your UI stays in sync with the underlying data, especially in interactive or dynamic applications.

8. Theming and Styling

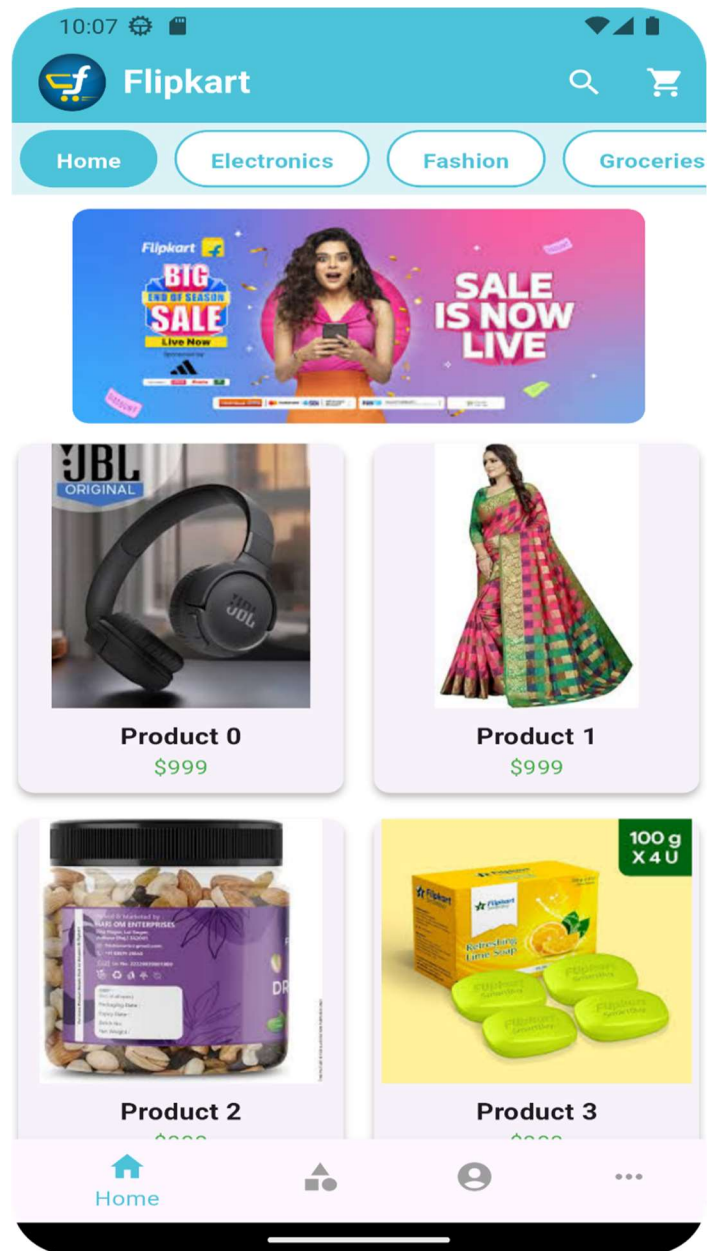
- Flutter allows you to define a global **Theme** for your app using **ThemeData**, which ensures consistent styling across the entire app.
- You can customize colors, typography, and button styles.

9. Animations and Transitions

- Flutter provides powerful animation support to create smooth and visually appealing transitions between UI states.
- **AnimatedContainer**: A widget that animates changes in properties like width, height, or color over a given duration.
- You can also create custom animations using **AnimationController** and **Tween**.

By utilizing these widgets and concepts, you can create a well-structured and visually appealing Flutter UI.

Screenshot:



Code Snippets: home_screen.dart

```
import 'package:flutter/material.dart';
import 'package:etsy_clone/theme.dart';
```

```
class HomeScreen extends
StatefulWidget {

  @override

  _HomeScreenState createState() =>
  _HomeScreenState();

}
```

```
class _HomeScreenState extends
State<HomeScreen> {

  bool isExpanded = false;

  TextEditingController searchController
= TextEditingController();

  List<Map<String, String>> suggestions
= [

    {"title": "Handmade Jewelry", "image":
"assets/images/home/bracelet.png"},

    {"title": "Personalized Gifts", "image":
"assets/images/home/personalizedgift.p
ng"},

    {"title": "Vintage Decor", "image":
"assets/images/home/stylishbags.png"},

    {"title": "Custom T-Shirts", "image":
"assets/images/home/ecofriendly.png"},

    {"title": "Resin Keychains", "image":
"assets/images/home/resinkeychain.png
"},

  ];

  List<Map<String, String>>
filteredSuggestions = [];

  String selectedImage = "";
```

```
@override

void dispose() {

  searchController.dispose();

  super.dispose();

}
```

```
@override

Widget build(BuildContext context) {

  return Scaffold(

    body: SingleChildScrollView(

      padding: EdgeInsets.all(16.0),

      child: Column(

        crossAxisAlignment:
CrossAxisAlignment.start,

        children: [

          _buildSearchBar(),

          if (selectedImage.isNotEmpty)
_buildSelectedImage(),

          SizedBox(height: 20),

          BannerSection(),

          SizedBox(height: 20),

          ShoppingSection(),

          SizedBox(height: 20),

          AdSection(),

        ],

      ),

    ),

  );

};
```

```

}

Widget _buildSearchBar() {
  return Column(
    children: [
      GestureDetector(
        onTap: () {
          setState(() {
            isExpanded = true;
          });
        },
        child: AnimatedContainer(
          duration: Duration(milliseconds:
300),
          padding:
EdgeInsets.symmetric(horizontal: 12,
vertical: 8),
          constraints: BoxConstraints(
            maxWidth:
MediaQuery.of(context).size.width *
0.95,
          ),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius:
BorderRadius.circular(25),
            boxShadow: [
              if (isExpanded)
                BoxShadow(
                  color:
Colors.black.withOpacity(0.2),
                  blurRadius: 5,

```

```

          offset: Offset(0, 3),
        ),
      ],
    ),
    child: Row(
      children: [
        Icon(Icons.search, color:
Colors.grey),
        SizedBox(width: 8),
        Expanded(
          child: isExpanded
            ? TextField(
                controller:
searchController,
                autofocus: true,
                decoration:
InputDecoration(
                  hintText: "Search for
anything...",
                  border:
InputBorder.none,
                ),
                onChanged: (query) {
                  setState(() {
                    filteredSuggestions =
suggestions
                      .where((item) =>
item["title"]!
                        .toLowerCase()
                          .contains(query.toLowerCase()))
                      .toList();

```

```

        ));
    },
)
: Text("Search for
anything...",
    style: TextStyle(color:
Colors.grey)),
),
if (isExpanded)
    GestureDetector(
        onTap: () {
            setState(() {
                isExpanded = false;
                searchController.clear();
                filteredSuggestions = [];
            });
        },
        child: Icon(Icons.close,
color: Colors.grey),
    ),
],
),
),
),
),
if (isExpanded &&
filteredSuggestions.isNotEmpty)
    Container(
        margin: EdgeInsets.only(top: 5),
        padding:
EdgeInsets.symmetric(horizontal: 10,
vertical: 5),

```

```

        decoration: BoxDecoration(
            color: Colors.white,
            borderRadius:
BorderRadius.circular(10),
            boxShadow: [
                BoxShadow(color:
Colors.black.withOpacity(0.1),
blurRadius: 5),
            ],
        ),
        child: Column(
            children:
filteredSuggestions.map((suggestion) {
                return ListTile(
                    leading:
Image.asset(suggestion["image"]!),
                    width: 40, height: 40, fit:
BoxFit.cover),
                    title: Text(suggestion["title"]!),
                    onTap: () {
                        setState(() {
                            searchController.text =
suggestion["title"]!;
                            selectedImage =
suggestion["image"]!;
                            isExpanded = false;
                            filteredSuggestions = [];
                        });
                    },
                );
            }).toList(),
        ),

```

```

        ),
    ],
);
}

Widget _buildSelectedImage() {
    return Padding(
        padding: const EdgeInsets.only(top:
10),
        child: ClipRRect(
            borderRadius:
BorderRadius.circular(10),
            child: Image.asset(selectedImage,
width: 150, height: 150, fit:
BoxFit.cover),
        ),
    );
}
}

```

```

class BannerSection extends
StatelessWidget {

    @override
    Widget build(BuildContext context) {
        return SizedBox(
            height: 140,
            child: ListView(
                scrollDirection: Axis.horizontal,
                padding: EdgeInsets.only(left: 10),
                children: [

```

```

                BannerTile(imagePath:
"assets/images/home/stichedsweters.pn
g", text: "Embroidered Kids Sweaters"),
                SizedBox(width: 10),
                BannerTile(imagePath:
"assets/images/home/personalizedgift.p
ng", text: "Personalized Gifts"),
                SizedBox(width: 10),
                BannerTile(imagePath:
"assets/images/home/bracelet.png",
text: "Jewellery"),
            ],
        ),
    );
}
}

```

```

class BannerTile extends StatefulWidget
{
    final String imagePath;
    final String text;

```

```

    BannerTile({required this.imagePath,
required this.text});

```

```

    @override
    _BannerTileState createState() =>
_BannerTileState();
}

```

```

class _BannerTileState extends
State<BannerTile> {
    bool isClicked = false;

```

```

@override
Widget build(BuildContext context) {
  return Container(
    width: 280,
    decoration: BoxDecoration(
      borderRadius:
        BorderRadius.circular(12),
      color: Colors.yellow.shade100,
    ),
    child: Row(
      children: [
        // Left: Text & Button
        Expanded(
          flex: 1,
          child: Padding(
            padding: EdgeInsets.all(10),
            child: Column(
              crossAxisAlignment:
                CrossAxisAlignment.start,
              mainAxisAlignment:
                MainAxisAlignment.center,
              children: [
                Text(
                  widget.text,
                  style: TextStyle(fontWeight:
                    FontWeight.bold, fontSize: 16),
                ),
                SizedBox(height: 10),
                ElevatedButton(
                  onPressed: () {

```

```

                    setState(() {
                      isClicked = !isClicked;
                    });
                },
                style:
                  ElevatedButton.styleFrom(
                    backgroundColor:
                      isClicked ? Colors.orange :
                      Colors.black,
                    padding:
                      EdgeInsets.symmetric(horizontal: 15,
                        vertical: 5),
                    shape:
                      RoundedRectangleBorder(
                        borderRadius:
                          BorderRadius.circular(5),
                      ),
                    ),
                    child: Text("Shop Now",
                      style: TextStyle(color: Colors.white)),
                ),
              ],
            ),
            ),
            ),
            ),
            ClipRRect(
              borderRadius:
                BorderRadius.circular(10),
              child:
                Image.asset(widget.imagePath, fit:
                  BoxFit.cover, height: 120, width: 120),
            ),
          ],
        ),

```

```

    );
  }
}

// 🛒 Shopping Section (Responsive Heart Icon)
class ShoppingSection extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: EdgeInsets.all(10),
      child: Column(
        crossAxisAlignment:
        CrossAxisAlignment.start,
        children: [
          Text(
            "Keep Shopping",
            style:
            Theme.of(context).textTheme.titleLarge,
          ),
          SizedBox(height: 10),
          SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            child: Row(
              children: [
                ShoppingTile(imagePath:
                "assets/images/home/woodenbox.png",
                label: "Recently Viewed"),
                SizedBox(width: 10),

```

```

                ShoppingTile(imagePath:
                "assets/images/home/resinkeychain.png",
                label: "Recently Favorited"),
                SizedBox(width: 10),
                ShoppingTile(imagePath:
                "assets/images/home/ecofriendly.png",
                label: "Eco-Friendly"),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

// 🛒 Modified Shopping Tile Widget
(Interactive Heart Icon)
class ShoppingTile extends StatefulWidget {
  final String imagePath;
  final String label;

  ShoppingTile({required this.imagePath,
    required this.label});

  @override
  _ShoppingTileState createState() =>
    _ShoppingTileState();
}

class _ShoppingTileState extends
  State<ShoppingTile> {
  bool isFavorited = false;

```

```

@override
Widget build(BuildContext context) {
  return Container(
    width: 160,
    child: Stack(
      children: [
        Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            ClipRRect(
              borderRadius:
BorderRadius.circular(10),
              child: Image.asset(
                widget.imagePath,
                fit: BoxFit.cover,
                height: 140,
                width: double.infinity,
              ),
            ),
            SizedBox(height: 5),
            Text(widget.label, style:
TextStyle(fontWeight: FontWeight.bold)),
          ],
        ),
        // Heart Icon for Wishlist
        Positioned(
          top: 10,
          right: 10,

```

```

child: GestureDetector(
  onTap: () {
    setState(() {
      isFavorited = !isFavorited;
    });
  },
  child: Icon(
    isFavorited ? Icons.favorite :
Icons.favorite_border,
    color: isFavorited ? const
Color(0xFFFF44336) : Colors.grey,
    size: 24,
  ),
),
),
);
}
}
// Ad Tile Widget
class AdTile extends StatelessWidget {
  final String imagePath;

  AdTile({required this.imagePath});

  @override
  Widget build(BuildContext context) {
    return Container(

```

```

width: 160,
child: ClipRRect(
  borderRadius:
    BorderRadius.circular(10),
  child: Image.asset(
    imagePath,
    fit: BoxFit.cover,
    height: 140,
    width: double.infinity,
  ),
),
);
}
}

// Ad Section

class AdSection extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: EdgeInsets.all(10),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          Text(
            "Because you viewed",
            style:
Theme.of(context).textTheme.titleMediu
m,

```

```

),
    Text("Including ads by Etsy
sellers", style: TextStyle(color:
Colors.grey, fontSize: 12)),
    SizedBox(height: 10),
    SingleChildScrollView(
      scrollDirection: Axis.horizontal,
      child: Row(
        children: [
          AdTile(imagePath:
"assets/images/home/resinkeychain.png
"),
          SizedBox(width: 10),
          AdTile(imagePath:
"assets/images/home/scentedcandles.p
ng"),
          SizedBox(width: 10),
          AdTile(imagePath:
"assets/images/home/stylishbags.png"),
        ],
      ),
    ),
  ],
),
);
}
}

```

```

    },
    child: AnimatedContainer(
      duration: Duration(milliseconds:
300),
      padding:
EdgeInsets.symmetric(horizontal: 12,
vertical: 8),
      constraints: BoxConstraints(
        maxWidth:
MediaQuery.of(context).size.width - 40,
      ),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius:
BorderRadius.circular(25),
        boxShadow: [
          if (isExpanded)
            BoxShadow(
              color:
Colors.black.withOpacity(0.2),
              blurRadius: 5,
              offset: Offset(0, 3),
            ),
        ],
      ),
    child: Row(
      children: [
        Icon(Icons.search, color:
Colors.grey),
        SizedBox(width: 8),
        Expanded(
          child: isExpanded
            ? TextField(
                controller:
searchController,
                autofocus: true,
                decoration:
InputDecoration(
                  hintText: "Search for
anything...",

```

```

        border:
InputBorder.none,
      ),
      onChanged: (query) {
        setState(() {
          filteredSuggestions =
suggestions
            .where((item) =>
item.toLowerCase().contains(query.toLo
werCase()))
              .toList();
        });
      },
    )
    : Text("Search for
anything...",
      style: TextStyle(color:
Colors.grey)),
  ),
  if (isExpanded)
    GestureDetector(
      onTap: () {
        setState(() {
          isExpanded = false;
          searchController.clear();
        });
      },
      child: Icon(Icons.close,
color: Colors.grey),
    ),
  ],
),
),
),
),
if (isExpanded &&
filteredSuggestions.isNotEmpty)
  Container(

```

```

margin: EdgeInsets.only(top: 5),
padding:
EdgeInsets.symmetric(horizontal: 10,
vertical: 5),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius:
BorderRadius.circular(10),
        boxShadow: [
          BoxShadow(color:
Colors.black.withOpacity(0.1),
blurRadius: 5),
        ],
      ),
      child: Column(
        children:
filteredSuggestions.map((suggestion) {
          return ListTile(
            title: Text(suggestion),
            onTap: () {
              setState(() {
                searchController.text =
suggestion;
                isExpanded = false;
              });
            },
          );
        }).toList(),
      ),
    ],
  );
}
}

```

MAD & PWA Lab

Journal

Experiment No.	03
Experiment Title.	To include icons, images, fonts in Flutter app
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

EXPERIMENT NO: 03

Aim:

To implement icons, images, and custom fonts in a Flutter application to enhance its visual aesthetics and user experience.

Theory:

Flutter is a powerful open-source UI framework that enables developers to create natively compiled applications for mobile, web, and desktop platforms from a single codebase. One of Flutter's key advantages is its ability to provide highly customizable user interfaces, making it easier to build visually appealing applications.

In app development, icons, images, and fonts play a crucial role in improving usability and design consistency. These elements help in conveying information effectively and reinforcing branding. Flutter offers seamless integration of these visual components, enhancing both functionality and user engagement.

Importance of Visual Elements in Flutter

- **Enhanced User Experience:** Icons and images make the app more intuitive and visually engaging.
- **Information Conveyance:** Icons provide quick representation, reducing the need for lengthy text descriptions.
- **Branding and Personalization:** Custom fonts and images create a unique brand identity, making the app stand out.

Flutter supports multiple image formats (JPEG, PNG, WebP, GIF, etc.) and allows the integration of icons and fonts effortlessly. Below are the methods to incorporate these elements into a Flutter application:

1. Adding Images in Flutter

Flutter supports **local** and **network images** for UI components.

Local Images

Images stored within the project can be used in the application by defining them in the asset folder and declaring them in the configuration file. Supported image formats include **PNG, JPG, GIF, SVG**, and more.

Network Images

Flutter allows fetching images from external sources using URLs. Cached images can be utilized for improved performance.

2. Adding Custom Fonts in Flutter

Custom fonts allow developers to enhance the UI design and branding of their applications.

Steps to Add Custom Fonts:

1. The font files need to be downloaded and stored in the project directory.
2. These fonts must be declared in the project configuration file.
3. Once declared, the fonts can be applied to text widgets for a distinct appearance.

3. Using Icons in Flutter

Icons play a crucial role in UI clarity and usability. Flutter supports both **built-in Material Icons** and **custom icons**.

Built-in Icons (Material Icons)

Flutter provides a collection of icons that are integrated within the framework, requiring no additional setup.

Custom Icons (SVG, PNG, Font Icons)

Developers can integrate custom icons in various formats, such as **SVG**, **PNG**, or **Font-Based Icons** like FontAwesome. These icons enhance visual consistency and allow for a personalized UI experience.

Code Snippets: home_screen.dart

```
import 'package:flutter/material.dart';
import 'package:etsy_clone/theme.dart';

class HomeScreen extends StatefulWidget {
  @override
  _HomeScreenState createState() =>
    _HomeScreenState();
}
```

```

class _HomeScreenState extends
State<HomeScreen> {
  bool isExpanded = false;
  TextEditingController searchController
= TextEditingController();
  List<Map<String, String>> suggestions
= [
    {"title": "Handmade Jewelry", "image":
"assets/images/home/bracelet.png"},
    {"title": "Personalized Gifts", "image":
"assets/images/home/personalizedgift.p
ng"},
    {"title": "Vintage Decor", "image":
"assets/images/home/stylishbags.png"},
    {"title": "Custom T-Shirts", "image":
"assets/images/home/ecofriendly.png"},
    {"title": "Resin Keychains", "image":
"assets/images/home/resinkeychain.png
"},
  ];
  List<Map<String, String>>
filteredSuggestions = [];
  String selectedImage = "";

```

```

@override
void dispose() {
  searchController.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SingleChildScrollView(
      padding: EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          _buildSearchBar(),

```

```

          if (selectedImage.isNotEmpty)
            _buildSelectedImage(),
            SizedBox(height: 20),
            BannerSection(),
            SizedBox(height: 20),
            ShoppingSection(),
            SizedBox(height: 20),
            AdSection(),
          ],
        ),
      ),
    );
}

```

```

Widget _buildSearchBar() {
  return Column(
    children: [
      GestureDetector(
        onTap: () {
          setState(() {
            isExpanded = true;
          });
        },
        child: AnimatedContainer(
          duration: Duration(milliseconds:
300),
          padding:
EdgeInsets.symmetric(horizontal: 12,
vertical: 8),
          constraints: BoxConstraints(
            maxWidth:
MediaQuery.of(context).size.width *
0.95,
          ),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius:
BorderRadius.circular(25),
            boxShadow: [
              if (isExpanded)

```

```

        BoxShadow(
          color:
Colors.black.withOpacity(0.2),
          blurRadius: 5,
          offset: Offset(0, 3),
        ),
      ],
    ),
    child: Row(
      children: [
        Icon(Icons.search, color:
Colors.grey),
        SizedBox(width: 8),
        Expanded(
          child: isExpanded
            ? TextField(
                controller:
searchController,
                autofocus: true,
                decoration:
InputDecoration(
                  hintText: "Search for
anything...",
                  border:
InputBorder.none,
                ),
                onChanged: (query) {
                  setState(() {
                    filteredSuggestions =
suggestions
                      .where((item) =>
item["title"]!
                        .toLowerCase()
                          .contains(query.toLowerCase()))
                      .toList();
                  });
                },
              )

```

```

          : Text("Search for
anything..."),
          style: TextStyle(color:
Colors.grey)),
        ),
        if (isExpanded)
          GestureDetector(
            onTap: () {
              setState(() {
                isExpanded = false;
                searchController.clear();
                filteredSuggestions = [];
              });
            },
            child: Icon(Icons.close,
color: Colors.grey),
          ),
        ],
      ),
    ),
    if (isExpanded &&
filteredSuggestions.isNotEmpty)
      Container(
        margin: EdgeInsets.only(top: 5),
        padding:
EdgeInsets.symmetric(horizontal: 10,
vertical: 5),
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius:
BorderRadius.circular(10),
          boxShadow: [
            BoxShadow(color:
Colors.black.withOpacity(0.1),
blurRadius: 5),
          ],
        ),
        child: Column(

```

```

        children:
filteredSuggestions.map((suggestion) {
    return ListTile(
        leading:
Image.asset(suggestion["image"]!,
        width: 40, height: 40, fit:
BoxFit.cover),
        title: Text(suggestion["title"]!),
        onTap: () {
            setState(() {
                searchController.text =
suggestion["title"]!;
                selectedImage =
suggestion["image"]!;
                isExpanded = false;
                filteredSuggestions = [];
            });
        },
    );
}).toList(),
),
],
);
}

Widget _buildSelectedImage() {
    return Padding(
        padding: const EdgeInsets.only(top:
10),
        child: ClipRRect(
            borderRadius:
BorderRadius.circular(10),
            child: Image.asset(selectedImage,
width: 150, height: 150, fit:
BoxFit.cover),
        ),
    );
}
}

```

```

class BannerSection extends
StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return SizedBox(
            height: 140,
            child: ListView(
                scrollDirection: Axis.horizontal,
                padding: EdgeInsets.only(left: 10),
                children: [
                    BannerTile(imagePath:
"assets/images/home/stichedsweters.png", text: "Embroidered Kids Sweaters"),
                    SizedBox(width: 10),
                    BannerTile(imagePath:
"assets/images/home/personalizedgift.png", text: "Personalized Gifts"),
                    SizedBox(width: 10),
                    BannerTile(imagePath:
"assets/images/home/bracelet.png",
text: "Jewellery"),
                ],
            ),
        );
    }
}

class BannerTile extends StatefulWidget {
    {
        final String imagePath;
        final String text;

        BannerTile({required this.imagePath,
required this.text});

        @override
        _BannerTileState createState() =>
        _BannerTileState();
    }
}

```

```

class _BannerTileState extends
State<BannerTile> {
  bool isClicked = false;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 280,
      decoration: BoxDecoration(
        borderRadius:
BorderRadius.circular(12),
        color: Colors.yellow.shade100,
      ),
      child: Row(
        children: [
          // Left: Text & Button
          Expanded(
            flex: 1,
            child: Padding(
              padding: EdgeInsets.all(10),
              child: Column(
                crossAxisAlignment:
CrossAxisAlignment.start,
                mainAxisAlignment:
MainAxisAlignment.center,
                children: [
                  Text(
                    widget.text,
                    style: TextStyle(fontWeight:
FontWeight.bold, fontSize: 16),
                  ),
                  SizedBox(height: 10),
                  ElevatedButton(
                    onPressed: () {
                      setState(() {
                        isClicked = !isClicked;
                      });
                    },
                    style:
ElevatedButton.styleFrom(

```

```

          backgroundColor:
isClicked ? Colors.orange :
Colors.black,
          padding:
EdgeInsets.symmetric(horizontal: 15,
vertical: 5),
          shape:
RoundedRectangleBorder(
            borderRadius:
BorderRadius.circular(5),
          ),
          child: Text("Shop Now",
style: TextStyle(color: Colors.white)),
        ],
      ),
    ),
    ClipRRect(
      borderRadius:
BorderRadius.circular(10),
      child:
Image.asset(widget.imagePath, fit:
BoxFit.cover, height: 120, width: 120),
    ),
  ],
);
}
}

```

```

// Shopping Section (Responsive Heart
Icon)
class ShoppingSection extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Padding(

```

```

padding: EdgeInsets.all(10),
child: Column(
  crossAxisAlignment:
CrossAxisAlignment.start,
  children: [
    Text(
      "Keep Shopping",
      style:
Theme.of(context).textTheme.titleLarge,
    ),
    SizedBox(height: 10),
    SingleChildScrollView(
      scrollDirection: Axis.horizontal,
      child: Row(
        children: [
          ShoppingTile(imagePath:
"assets/images/home/woodenbox.png",
label: "Recently Viewed"),
          SizedBox(width: 10),
          ShoppingTile(imagePath:
"assets/images/home/resinkeychain.png
", label: "Recently Favorited"),
          SizedBox(width: 10),
          ShoppingTile(imagePath:
"assets/images/home/ecofriendly.png",
label: "Eco-Friendly"),
        ],
      ),
    ),
  ],
);
}
}

```

```

// Shopping Tile Widget (Interactive
Heart Icon)
class ShoppingTile extends
StatefulWidget {
  final String imagePath;

```

```

  final String label;

  ShoppingTile({required this.imagePath,
required this.label});

  @override
  _ShoppingTileState createState() =>
  _ShoppingTileState();
}

class _ShoppingTileState extends
State<ShoppingTile> {
  bool isFavorited = false;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 160,
      child: Stack(
        children: [
          Column(
            crossAxisAlignment:
CrossAxisAlignment.start,
            children: [
              ClipRRect(
                borderRadius:
BorderRadius.circular(10),
                child: Image.asset(
                  widget.imagePath,
                  fit: BoxFit.cover,
                  height: 140,
                  width: double.infinity,
                ),
              ),
              SizedBox(height: 5),
              Text(widget.label, style:
TextStyle(fontWeight: FontWeight.bold)),
            ],
          ),
          // Heart Icon for Wishlist

```

```

        Positioned(
          top: 10,
          right: 10,
          child: GestureDetector(
            onTap: () {
              setState(() {
                isFavorited = !isFavorited;
              });
            },
            child: Icon(
              isFavorited ? Icons.favorite :
Icons.favorite_border,
              color: isFavorited ? const
Color(0xFFFF44336) : Colors.grey,
              size: 24,
            ),
          ),
        ],
      ),
    );
  }
}
// Ad Tile Widget
class AdTile extends StatelessWidget {
  final String imagePath;

  AdTile({required this.imagePath});

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 160,
      child: ClipRRect(
        borderRadius:
BorderRadius.circular(10),
        child: Image.asset(
          imagePath,
          fit: BoxFit.cover,
          height: 140,

```

```

          width: double.infinity,
        ),
      ),
    );
  }
}

// Ad Section
class AdSection extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: EdgeInsets.all(10),
      child: Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
          Text(
            "Because you viewed",
            style:
Theme.of(context).textTheme.titleMediu
m,
          ),
          Text("Including ads by Etsy
sellers", style: TextStyle(color:
Colors.grey, fontSize: 12)),
          SizedBox(height: 10),
          SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            child: Row(
              children: [
                AdTile(imagePath:
"assets/images/home/resinkeychain.png
"),
                SizedBox(width: 10),
                AdTile(imagePath:
"assets/images/home/scentedcandles.p
ng"),
                SizedBox(width: 10),

```

```

        AdTile(imagePath:
"assets/images/home/stylishbags.png"),
      ],
    ),
  ],
),
);
}
}

```

Deals_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

class DealsPage extends
StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          "🔥 Exclusive Deals &
🔥 Coupons",
          style: TextStyle(color: const
Color.fromARGB(255, 231, 64, 42),
fontWeight: FontWeight.bold),

```

```

        ),
        backgroundColor: Colors.white,
        elevation: 1,
        iconTheme: IconThemeData(color:
Colors.black),
      ),
      body: Padding(
        padding: EdgeInsets.all(12),
        child: SingleChildScrollView(
          child: Column(
            children: [
              CouponCard(
                discount: "20% OFF",
                code: "SAVE20",
                expiry: "Expires: Feb 29,
2025",
                gradientColors:
[Colors.orange.shade300,
Colors.orange.shade700],
              ),
              SizedBox(height: 15),
              CouponCard(
                discount: "Flat ₹100 OFF", code:
"FLAT100",
                expiry: "Expires: March 10,
2025",
                gradientColors:
[Colors.blue.shade300,
Colors.blue.shade700],
              ),
              SizedBox(height: 15),
              CouponCard(
                discount: "Buy 1 Get 1 Free",
                code: "BOGO",
                expiry: "Expires: March 25,
2025",
                gradientColors:
[Colors.green.shade300,
Colors.green.shade700],
              ),

```

```

    ],
  ),
),
),
);
}
}

```

```

class CouponCard extends
StatelessWidget {
  final String discount;
  final String code;
  final String expiry;
  final List<Color> gradientColors;

```

```

  CouponCard({
    required this.discount,
    required this.code,
    required this.expiry,
    required this.gradientColors,
  });

```

```

  void copyToClipboard(BuildContext
context, String text) {

```

```

    Clipboard.setData(ClipboardData(text:
text));

```

```

    ScaffoldMessenger.of(context).showSna
ckBar(
      SnackBar(
        content: Row(
          children: [
            Icon(Icons.check_circle, color:
Colors.white, size: 20),
            SizedBox(width: 8),
            Text("Coupon Code Copied:
$text", style: TextStyle(fontWeight:
FontWeight.bold)),
          ],

```

```

        ),
        backgroundColor: Colors.black87,
        duration: Duration(seconds: 2),
      ),
    );
  }

```

```

@override
Widget build(BuildContext context) {
  return Container(
    width: double.infinity,
    padding: EdgeInsets.all(15),
    decoration: BoxDecoration(
      gradient: LinearGradient(
        colors: gradientColors,
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ),
      borderRadius:
BorderRadius.circular(12),
      boxShadow: [
        BoxShadow(color:
Colors.grey.shade400, blurRadius: 8,
spreadRadius: 2),
      ],
    ),
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        Text(
          discount,
          style: TextStyle(fontSize: 22,
fontWeight: FontWeight.bold, color:
Colors.white),
        ),
        SizedBox(height: 8),
        Divider(
          color: Colors.white54,
          thickness: 1.2,

```



```

        {"title": "Craft Supplies & Tools",
"image":
"assets/images/shop/craft.png"},
        {"title": "Toys & Games", "image":
"assets/images/shop/toys.png"},
        {"title": "Art & Collectibles", "image":
"assets/images/shop/giftideas.png"},
        {"title": "Paper & Party Supplies",
"image":
"assets/images/shop/cards.png"},
        {"title": "Accessories", "image":
"assets/images/shop/jewellery.png"},
        {"title": "Bags & Purses", "image":
"assets/images/shop/purse.png"},
        {"title": "Books, Movies & Music",
"image":
"assets/images/shop/bookcover.png"},
        {"title": "Bath & Beauty", "image":
"assets/images/shop/bath.png"},
        {"title": "Weddings", "image":
"assets/images/shop/wedding.png"},
];

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor:
Colors.grey.shade100, // Light pastel
background
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      toolbarHeight: 70, // Slightly taller
AppBar
      title: Container(
        padding:
EdgeInsets.symmetric(horizontal: 15),
        decoration: BoxDecoration(
          color: Colors.grey.shade200, //
Light grey background

```

```

borderRadius:
BorderRadius.circular(30),
    boxShadow: [
      BoxShadow(
        color: Colors.grey.shade300,
        blurRadius: 6,
        spreadRadius: 1,
        offset: Offset(0, 3),
      ),
    ],
  ),
  child: Row(
    children: [
      Icon(Icons.search, color:
Colors.grey.shade600), // Search icon
      SizedBox(width: 10),
      Expanded(
        child: TextField(
          decoration: InputDecoration(
            hintText: "Search for
anything on Etsy",
            border: InputBorder.none,
            hintStyle: TextStyle(color:
Colors.grey.shade600),
          ),
        ),
      ),
      Icon(Icons.camera_alt, color:
Colors.grey.shade600), // Camera icon
    ],
  ),
  body: Padding(
    padding: const
EdgeInsets.all(12.0),
    child: GridView.builder(
      itemCount: categories.length,

```

```

        gridDelegate:
SliverGridDelegateWithFixedCrossAxis
Count(
    crossAxisCount: 2,
    childAspectRatio: 3.5,
    crossAxisSpacing: 12,
    mainAxisSpacing: 12,
),
    itemBuilder: (context, index) {
    return GestureDetector(
    onTap: () {
        // Add your navigation or
action here
    },
    child: AnimatedContainer(
        duration:
Duration(milliseconds: 300),
        decoration: BoxDecoration(
            color: Colors.white,
            borderRadius:
BorderRadius.circular(15),
            boxShadow: [
                BoxShadow(
                    color:
Colors.grey.shade300,
                    blurRadius: 8,
                    spreadRadius: 2,
                    offset: Offset(0, 4),
                ),
            ],
        ),
        child: Row(
            ),
        ),
    ],
),
);
},
),

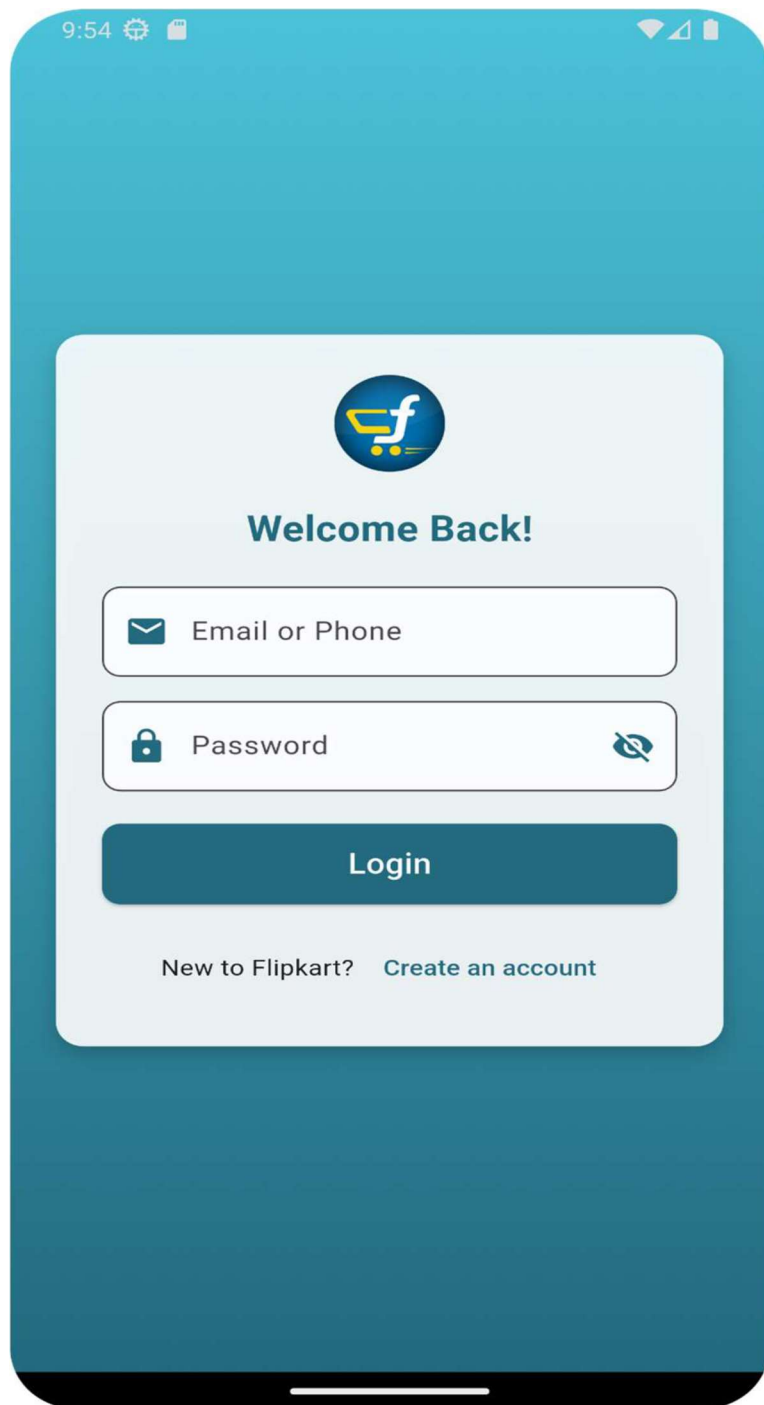
```

```

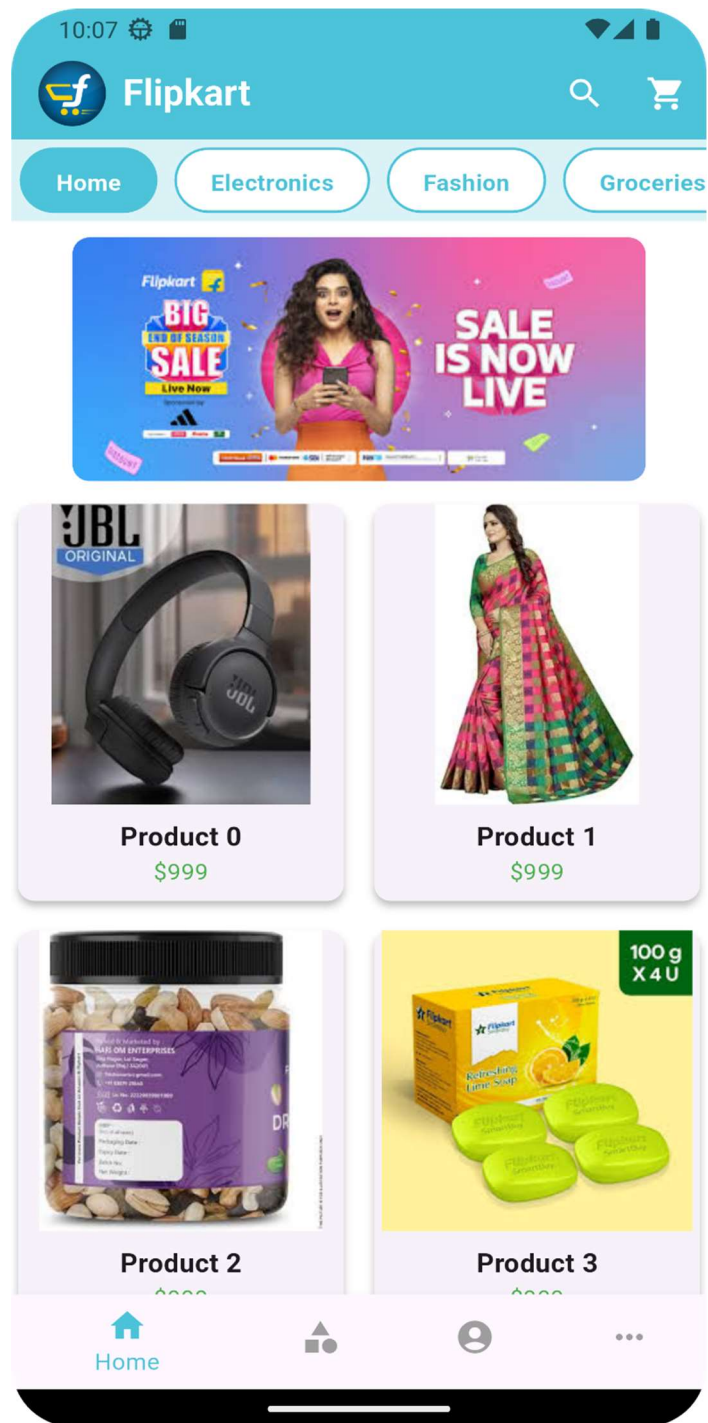
        children: [
            Expanded(
                child: Padding(
                    padding: const
EdgeInsets.symmetric(horizontal: 12.0),
                    child: Text(
categories[index]["title"]!,
                        style: TextStyle(
                            fontSize: 16,
                            fontWeight:
FontWeight.w600,
                            color: Colors.black87,
                        ),
                    ),
                ),
                child: ClipRRect(
                    borderRadius:
BorderRadius.only(
                        topRight:
Radius.circular(15),
                        bottomRight:
Radius.circular(15),
                    ),
                    child: Image.asset(
categories[index]["image"]!,
                        fit: BoxFit.cover,
                        width: 90,
                        height: double.infinity,
                    ),
                ),
            ),
        ],
    );
}
}

```

Screenshots:



0



11:46



Create Account



Full Name



Email



Password

Sign Up

Already have an account? [Login](#)

MAD & PWA Lab

Journal

Experiment No.	04
Experiment Title.	To create an interactive Form using form widget
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

EXPERIMENT NO: 04

Name: Aditi Taksale

Class: D15A

Roll No: 58

Aim: To create an interactive Form using form widget

Theory:

Flutter provides a Form widget that enables developers to create interactive forms for collecting user input. Forms are an essential part of many applications, such as registration screens, login pages, feedback forms, and data entry modules. They allow structured input gathering, validation, and submission, enhancing user interaction and data integrity.

Flutter's Form widget works in combination with various input fields, validation techniques, and state management to ensure a seamless user experience.

Importance of Forms in App Development

1. Efficient User Input Handling – Forms streamline the process of collecting user data.
2. Validation Mechanism – Ensures only correct and meaningful data is submitted.
3. State Management – Helps in tracking and modifying input fields dynamically.
4. Improved User Experience – Provides structured input fields for better accessibility and usability.

Key Components of Forms in Flutter

1. Form Widget (Form)

The Form widget acts as a container that manages multiple input fields. It provides an easy way to validate and track changes in form fields.

Key Properties:

- key – A GlobalKey<FormState> to track form state and validation.
- child – Contains form fields like TextFormField, DropdownButtonFormField, etc.

- `autovalidateMode` – Determines when validation messages should appear (disabled, always, onUserInteraction).

2. Input Fields (Form Fields)

a) TextFormField

- A fundamental widget for text input in Flutter.
- Supports validation, formatting, and user interaction.
- Properties:
 - `controller` – Controls and retrieves the text entered by the user.
 - `keyboardType` – Defines the type of input (text, number, email, etc.).
 - `decoration` – Customizes the appearance (label, hint text, icon).
 - `validator` – Implements validation logic.

b) DropdownButtonFormField

- Provides a dropdown list for selecting predefined options.
- Properties:
 - `items` – Defines the list of selectable options.
 - `value` – Holds the currently selected item.
 - `onChanged` – Executes when the user selects a different item.

c) CheckboxListTile

- Displays a checkbox with a title and subtitle.
- Properties:
 - `value` – Represents whether the checkbox is selected.
 - `onChanged` – Detects changes in the selection state.
 - `title` – Describes the purpose of the checkbox.

d) RadioListTile

- Provides a list of radio buttons where only one can be selected.
- Properties:
 - `value` – The value assigned to each radio button.
 - `groupValue` – The currently selected value within the group.
 - `onChanged` – Updates the selection when a new option is chosen.

e) SwitchListTile

- Allows users to toggle between on/off states.
- Properties:

- value – Represents the current state (true/false).
- onChanged – Detects when the switch is toggled.
- title – Displays the label for the switch.

3. Managing Form State

a) GlobalKey<FormState>

- A unique key used to track the form's state.
- Helps in validation and resetting form fields.

b) FormState Methods

- validate() – Checks if all fields meet the validation rules.
- save() – Saves the form data when validation is successful.
- reset() – Clears all fields in the form.

Form Validation and Submission Process

1. The user enters data into form fields.
2. On submission, FormState.validate() ensures all inputs are correct.
3. If valid, the save() function processes and stores the data.
4. If invalid, appropriate error messages are displayed.

Code Snippets: welcome_screen.dart

```
import 'package:flutter/material.dart';

import 'signup_screen.dart';

class WelcomeScreen extends
StatefulWidget {

  @override

  _WelcomeScreenState createState()
=> _WelcomeScreenState();

}
```

```
class _WelcomeScreenState extends
State<WelcomeScreen> with
SingleTickerProviderStateMixin {

  final TextEditingController
_emailController =
TextEditingController();

  final GlobalKey<FormState> _formKey
= GlobalKey<FormState>();

  bool isClicked = false;
```

```

late AnimationController
_shakeController;

late Animation<double>
_shakeAnimation;

@override
void initState() {
    super.initState();

    _shakeController =
    AnimationController(
        vsync: this,
        duration: Duration(milliseconds:
400),
    );

    _shakeAnimation =
    Tween<double>(begin: 0, end:
10).chain(CurveTween(curve:
Curves.elasticIn)).animate(_shakeContr
oller);
}

@override
void dispose() {
    _emailController.dispose();

    _shakeController.dispose();

    super.dispose();
}

```

```

// Email Validation Function

String? validateEmail(String? value) {
    if (value == null || value.isEmpty) {
        return "Please enter an email.";
    }

    String emailPattern =
r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a
-zA-Z]{2,}$';

    if
(!RegExp(emailPattern).hasMatch(value
)) {
        return "Enter a valid email address.";
    }

    return null;
}

void _onSignUpTap() {
    if (_formKey.currentState!.validate()) {
        setState(() {
            isClicked = true;
        });

        // Navigate to SignUpPage with
Email

```

```

Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) =>
SignUpPage(email:
_emailController.text),
  ),
);
} else {
  // Trigger shake animation on invalid
email
  _shakeController.forward(from: 0);
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor:
Color(0xFFFF3E5D0),
    body: Padding(
      padding: EdgeInsets.all(20),
      child: Column(
        mainAxisAlignment:
MainAxisAlignment.center,

```

```

children: [
  Icon(Icons.card_giftcard, size:
80, color: Color(0xFF7D5A50)),
  SizedBox(height: 10),
  Text(
    'Welcome to Etsy Clone',
    style: TextStyle(fontSize: 28,
fontWeight: FontWeight.bold, color:
Color(0xFF7D5A50)),
  ),
  SizedBox(height: 10),
  Text(
    'Discover unique handmade &
vintage items from talented sellers!',
    textAlign: TextAlign.center,
    style: TextStyle(fontSize: 16,
color: Colors.brown.shade700),
  ),
  SizedBox(height: 20),
  // Email Input Field with
Validation
  Form(
    key: _formKey,
    child: AnimatedBuilder(
      animation: _shakeAnimation,

```

```

        builder: (context, child) {
          return Transform.translate(
            offset:
Offset(_shakeAnimation.value, 0),
            child: child,
          );
        },
        child: TextFormField(
          controller: _emailController,
          keyboardType:
TextInputType.emailAddress,
          decoration: InputDecoration(
            labelText: "Enter your
email",
            border:
OutlineInputBorder(),
            prefixIcon:
Icon(Icons.email, color:
Color(0xFF7D5A50)),
          ),
          validator: validateEmail,
        ),
      ),
    ),
  ),
  SizedBox(height: 20),

```

```

// Sign Up Button
GestureDetector(
  onTap: _onSignUpTap,
  child: AnimatedContainer(
    duration:
Duration(milliseconds: 300),
    width: 200,
    height: 50,
    decoration: BoxDecoration(
      color: isClicked ?
Color(0xFFE67E22) :
Color.fromARGB(255, 159, 75, 31),
      borderRadius:
BorderRadius.circular(25),
      boxShadow: [
        BoxShadow(color:
Colors.black26, blurRadius: 6, offset:
Offset(0, 3)),
      ],
    ),
    child: Center(
      child: Text(
        "Sign Up",
        style: TextStyle(fontSize:
18, fontWeight: FontWeight.w600, color:
Colors.white),
      ),
    ),
  ),

```

```

        ),
      ),
    ),
  ],
),
),
);
}
}

```

Signup_screen.dart

```

import 'package:flutter/material.dart';

import 'main_navigation.dart'; // Import
the main navigation

```

```

class SignUpPage extends
StatefulWidget {

  final String email;

  SignUpPage({required this.email});

  @override
  _SignUpPageState createState() =>
  _SignUpPageState();

}

```

```

class _SignUpPageState extends
State<SignUpPage> {

```

```

  final _formKey =
GlobalKey<FormState>();

```

```

  final TextEditingController
_passwordController =
TextEditingController();

```

```

  bool _isButtonClicked = false;

```

```

  // Password validation conditions

```

```

  bool hasMinLength = false;

```

```

  bool hasUpperCase = false;

```

```

  bool hasNumber = false;

```

```

  bool hasSpecialChar = false;

```

```

  // Function to check password
conditions

```

```

  void _validatePassword(String value) {

```

```

    setState(() {

```

```

      hasMinLength = value.length >= 8;

```

```

      hasUpperCase =
value.contains(RegExp(r'[A-Z]'));

```

```

      hasNumber =
value.contains(RegExp(r'\d'));

```

```

        hasSpecialChar =
value.contains(RegExp(r'[!@#$$%^&*(),.
?":{}|<>]'));

    });
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Sign In to Etsy'),
      backgroundColor: Colors.white,
      elevation: 0,
      leading: IconButton(
        icon: Icon(Icons.arrow_back,
color: Colors.black),
        onPressed: () {
          Navigator.pop(context);
        },
      ),
    ),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,

```

```

child: Column(
  crossAxisAlignment:
CrossAxisAlignment.start,
  children: [
    // User Email Display
    Row(
      children: [
        Icon(Icons.account_circle,
size: 40, color: Colors.grey.shade700),
        SizedBox(width: 10),
        Text(
          widget.email,
          style: TextStyle(fontSize:
18, fontWeight: FontWeight.w500),
        ),
      ],
    ),
    SizedBox(height: 5),
    TextButton(
      onPressed: () {},
      child: Text("Use a different
email address", style: TextStyle(color:
Colors.blue)),
    ),

```

```

        SizedBox(height: 15),

        // Title
        Text(
            'Create your account',
            style: TextStyle(fontSize: 24,
fontWeight: FontWeight.bold),
        ),
        SizedBox(height: 5),
        Text("Registration is easy.",
style: TextStyle(fontSize: 16)),

        SizedBox(height: 20),

        // First Name Field
        TextFormField(
            decoration: InputDecoration(
                labelText: 'First name *',
                border: OutlineInputBorder(),
            ),
            validator: (value) {
                if (value == null ||
value.isEmpty) {
                    return 'Please enter your
first name';

```

```

        }
        return null;
    },
),

        SizedBox(height: 15),

        // Password Field
        TextFormField(
            controller:
_passwordController,
            decoration: InputDecoration(
                labelText: 'Password *',
                border: OutlineInputBorder(),
            ),
            obscureText: true,
            onChanged:
_validatePassword,
            validator: (value) {
                if (value == null ||
value.isEmpty) {
                    return 'Please enter your
password';
                }

```

```

        if (!hasMinLength ||
!hasUpperCase || !hasNumber ||
!hasSpecialChar) {

            return 'Password does not
meet the criteria';

        }

        return null;

    },

),

// Password Condition Hints

SizedBox(height: 10),

Column(

    crossAxisAlignment:
CrossAxisAlignment.start,

    children: [

        _passwordHint("At least 8
characters", hasMinLength),

        _passwordHint("At least one
uppercase letter", hasUpperCase),

        _passwordHint("At least one
number", hasNumber),

        _passwordHint("At least one
special character", hasSpecialChar),

    ],

),

```

```

SizedBox(height: 20),

// Terms & Conditions

RichText(

    text: TextSpan(

        style: TextStyle(color:
Colors.black, fontSize: 14),

        children: [

            TextSpan(text: "By clicking
Register, you agree to Etsy's "),

            TextSpan(

                text: "Terms of Use",

                style: TextStyle(color:
Colors.blue, fontWeight:
FontWeight.bold),

            ),

            TextSpan(text: " and "),

            TextSpan(

                text: "Privacy Policy.",

                style: TextStyle(color:
Colors.blue, fontWeight:
FontWeight.bold),

            ),

        ],

    ),

),

```

```

        SizedBox(height: 20),

        // Register Button
        Center(
            child: ElevatedButton(
                style:
                ElevatedButton.styleFrom(
                    backgroundColor:
                    _isButtonClicked ? Color(0xFFFF16521) :
                    Colors.grey.shade400,

                    minimumSize:
                    Size(double.infinity, 50),

                ),

                onPressed: () {

                    if
                    (_formKey.currentState!.validate()) {

                        setState(() {

                            _isButtonClicked = true;

                        });

                        // Navigate to Main
                        Navigation with Bottom Navigation Bar

                        Future.delayed(Duration(milliseconds:
                        500), () {

```

```

Navigator.pushReplacement(
                    context,

                    MaterialPageRoute(builder: (context) =>
                    MainNavigation()), // Change to
                    MainNavigation

                    );

                });

            }

        },

        child: Text('Register', style:
        TextStyle(color: Colors.white)),

        ),

    ),

    SizedBox(height: 10),

    // Communication Note

    Text(

        "Etsy may send you
        communications; you may change your
        preferences in your account settings.
        We'll never post without your
        permission.",

        style: TextStyle(fontSize: 12,
        color: Colors.grey.shade700),

    ),

```

```

    ],
  ),
),
),
);
}

```

// Helper Widget to Show Password Rules

```

Widget _passwordHint(String text, bool
isValid) {

```

```

  return Padding(

```

```

    padding: const EdgeInsets.only(left:
10, bottom: 5),

```

```

    child: Row(

```

```

      children: [

```

```

        Icon(isValid ? Icons.check_circle :
Icons.circle, size: 12, color: isValid ?
Colors.green : Colors.grey),

```

```

        SizedBox(width: 5),

```

```

        Text(

```

```

          text,

```

```

          style: TextStyle(fontSize: 12,
color: isValid ? Colors.green :
Colors.grey),

```

```

        ),

```

```

    ],
  ),
);
}
}

```

Checkout_form.dart

```

import 'package:flutter/material.dart';

```

```

class CheckoutForm extends
StatefulWidget {

```

```

  @override

```

```

  _CheckoutFormState createState() =>
_CheckoutFormState();

```

```

}

```

```

class _CheckoutFormState extends
State<CheckoutForm> {

```

```

  final _formKey =
GlobalKey<FormState>();

```

```

  String name = "", phone = "", address =
"", city = "", state = "", pin = "",
paymentMethod = 'Card';

```

```

  String personalizedMessage = "";

```

```

  int quantity = 1;

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Checkout', style:
TextStyle(color: Colors.orange,
fontWeight: FontWeight.bold)),
      backgroundColor: Colors.white,
      iconTheme: IconThemeData(color:
Colors.orange),
    ),
    body: Padding(
      padding: EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: ListView(
          children: [
            buildTextField('Full Name',
(value) => name = value!),
            buildTextField('Phone Number',
(value) => phone = value!),
            buildTextField('Address', (value)
=> address = value!),
            buildTextField('City', (value) =>
city = value!),
            buildTextField('State', (value)
=> state = value!),

```

```

      buildTextField('PIN Code',
(value) => pin = value!),
      buildDropdown(),
      buildTextField('Personalized
Message (Optional)', (value) =>
personalizedMessage = value!),
      buildQuantityField(),
      SizedBox(height: 20),
      ElevatedButton(
        style:
ElevatedButton.styleFrom(
          backgroundColor:
Colors.orange,
          shape:
RoundedRectangleBorder(borderRadius
: BorderRadius.circular(8)),
        ),
        onPressed: () {
          if
(_formKey.currentState!.validate()) {
            _formKey.currentState!.save();
            print('Order placed with:
Name: $name, Address: $address,
Payment: $paymentMethod');
          }
        },

```

```

        child: Text('Place Order', style:
TextStyle(color: Colors.white, fontSize:
16, fontWeight: FontWeight.bold)),

    ),

  ],

),

),

),

);

}

```

```

Widget buildTextField(String label,
Function(String onSave) {

```

```

    return Padding(

      padding:
EdgeInsets.symmetric(vertical: 8),

      child: TextFormField(

        decoration: InputDecoration(

          labelText: label,

          border:
OutlineInputBorder(borderRadius:
BorderRadius.circular(8)),

        ),

        validator: (value) => value!.isEmpty
? 'Enter $label' : null,

        onSave: (value) =>
onSave(value!),

```

```

    ),

  );

}

Widget buildDropdown() {

  return Padding(

    padding:
EdgeInsets.symmetric(vertical: 8),

    child: DropdownButtonFormField(

      decoration: InputDecoration(

        labelText: 'Payment Method',

        border:
OutlineInputBorder(borderRadius:
BorderRadius.circular(8)),

      ),

      value: paymentMethod,

      items: ['Card', 'UPI', 'Cash on
Delivery'].map((method) {

        return DropdownMenuItem(value:
method, child: Text(method));

      }).toList(),

      onChanged: (value) => setState(()
=> paymentMethod = value!),

    ),

  );

}

```

```
Widget buildQuantityField() {  
  return Padding(  
    padding:  
EdgeInsets.symmetric(vertical: 8),  
    child: TextFormField(  
      decoration: InputDecoration(  
        labelText: 'Quantity',  
        border:  
OutlineInputBorder(borderRadius:  
BorderRadius.circular(8)),  
      ),  
      keyboardType:  
TextInputType.number,  
      initialValue: '1',  onSaved: (value)  
=> quantity = int.parse(value!),  
    ),  
  );  
}  
}
```

Screenshots:

11:43



My Cart



Home Product 3

₹999

Select Payment Method:



UPI (Not Available)



Cash on Delivery (COD)

Total: ₹999.0

Place Order

11:42



user Name



Home



My Orders



Received Orders



History



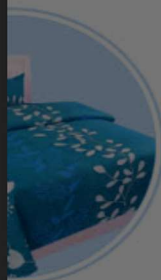
Search



Sign Out



Groceries



Product 1
99



Product 2
99



MAD & PWA Lab

Journal

Experiment No.	05
Experiment Title.	To apply navigation, routing and gestures in Flutter App
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

EXPERIMENT NO: 05

Name: Aditi Taksale

Class: D15A

Roll No: 58

Aim: To implement navigation, routing, and gestures in a Flutter application

Theory:

Navigation & Routing in Flutter

Navigation is the process of moving between different screens (or pages) in an app. Since Flutter follows a **widget-based architecture**, each screen is represented as a widget, and navigation is handled using a **stack-based system** (similar to how web browsers manage page history).

Navigation Approaches in Flutter:

1. **Imperative Navigation (Push-Pop Model)**
 - Uses `Navigator.push()` to open a new screen.
 - Uses `Navigator.pop()` to return to the previous screen.
 - Works like a stack (Last In, First Out - LIFO).
2. **Declarative Navigation (Named Routes & GoRouter)**
 - Uses predefined route names to navigate.
 - Helps in managing complex app navigation efficiently.
3. **Navigation with State Management**
 - Used in large-scale applications where navigation state is maintained using providers like **Provider, Riverpod, Bloc, or GetX**.
4. **Deep Linking & Dynamic Routing**
 - Allows external URLs to open specific screens within the app.
 - Useful for handling notifications and web links.

Routing in Flutter

Routing determines how users navigate through different sections of an application. It enables:

- **Defining and managing screens efficiently.**
- **Enhancing UX by ensuring smooth transitions.**
- **Passing and receiving data between screens.**

Types of Routing:

- **Basic Routing:** Manually navigating between screens using the Navigator class.
- **Named Routing:** Using a routes map to define screen names and their respective widgets.
- **On-Generate Routing:** Dynamically controlling navigation flow using conditions.
- **Nested Navigation:** Managing multiple navigation flows within tabs or bottom navigation bars.

Gestures in Flutter

Gestures allow users to interact with the app through touch-based actions, providing a more engaging user experience. Flutter has a powerful **GestureDetector** widget that helps recognize and handle different gestures.

Common Gesture Types:

- **Tap Gesture:** Used for buttons, links, and UI interactions.
- **Double Tap:** Often used for liking content or zooming.
- **Long Press:** Triggers additional options or context menus.
- **Swipe & Drag:** Used in carousels, sliders, and scrollable content.
- **Pinch & Zoom:** Common in images and maps.

Gesture Handling Techniques:

- **Using GestureDetector:** To detect gestures manually.
- **Using InkWell or InkResponse:** Provides visual feedback for tappable elements.
- **Custom Gesture Recognition:** Flutter allows combining multiple gestures using GestureRecognizer.

Importance of Navigation, Routing, and Gestures in Flutter Apps

- **User-Friendly Experience** – Navigation helps users seamlessly explore the app.
- **Efficient State Management** – Proper routing keeps the app organized.
- **Enhanced Interactivity** – Gestures make the app feel intuitive and engaging.
- **Optimized Performance** – Declarative navigation and lazy loading improve performance.
- **Platform Consistency** – Flutter's navigation system works smoothly on Android, iOS, and the web.

Code Snippet: main_navigation.dart

```
import 'package:flutter/material.dart';
import 'package:etsy_clone/theme.dart';
import 'home_screen.dart';
import 'deals_screen.dart';
import 'shop_screen.dart';
import 'favorites_screen.dart';
import 'cart_screen.dart';
```

```
void main() {
  runApp(EtsyCloneApp());
}
```

```
class EtsyCloneApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainNavigation(),
    );
  }
}
```

```
class MainNavigation extends StatefulWidget {
  @override
  _MainNavigationState createState() => _MainNavigationState();
}
```

```
class _MainNavigationState extends State<MainNavigation> {
  int _selectedIndex = 0;
```

```
  final List<Widget> _screens = [
    HomeScreen(),
    DealsPage(),
    ShopPage(),
    FavoritesScreen(favoriteItems: []),
    CartScreen(),
  ];
```

```
  void _onItemTapped(int index) {
    setState(() {
```

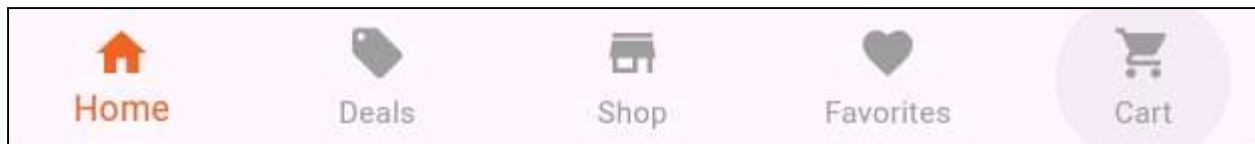
```

    _selectedIndex = index;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: _screens[_selectedIndex],
    bottomNavigationBar: BottomNavigationBar(
      type: BottomNavigationBarType.fixed,
      selectedItemColor: AppColors.primary,
      unselectedItemColor: AppColors.textSecondary,
      currentIndex: _selectedIndex,
      onTap: _onItemTapped,
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
        BottomNavigationBarItem(icon: Icon(Icons.local_offer), label: "Deals"),
        BottomNavigationBarItem(icon: Icon(Icons.store), label: "Shop"),
        BottomNavigationBarItem(icon: Icon(Icons.favorite), label: "Favorites"),
        BottomNavigationBarItem(icon: Icon(Icons.shopping_cart), label: "Cart"),
      ],
    ),
  );
}

```

This main_navigation.dart file is responsible for navigation between pages from the bottom navigation bar



Navigation Routing

- Splash screen to welcome screen

```
Future.delayed(Duration(seconds: 3), () {
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (context) => WelcomeScreen()),
  );
});
```

- Banner tile in home screen to cart screen

```
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => CartScreen()),
    );
  },
  child: Text("Shop Now", style: TextStyle(color: Colors.white)),
);
```

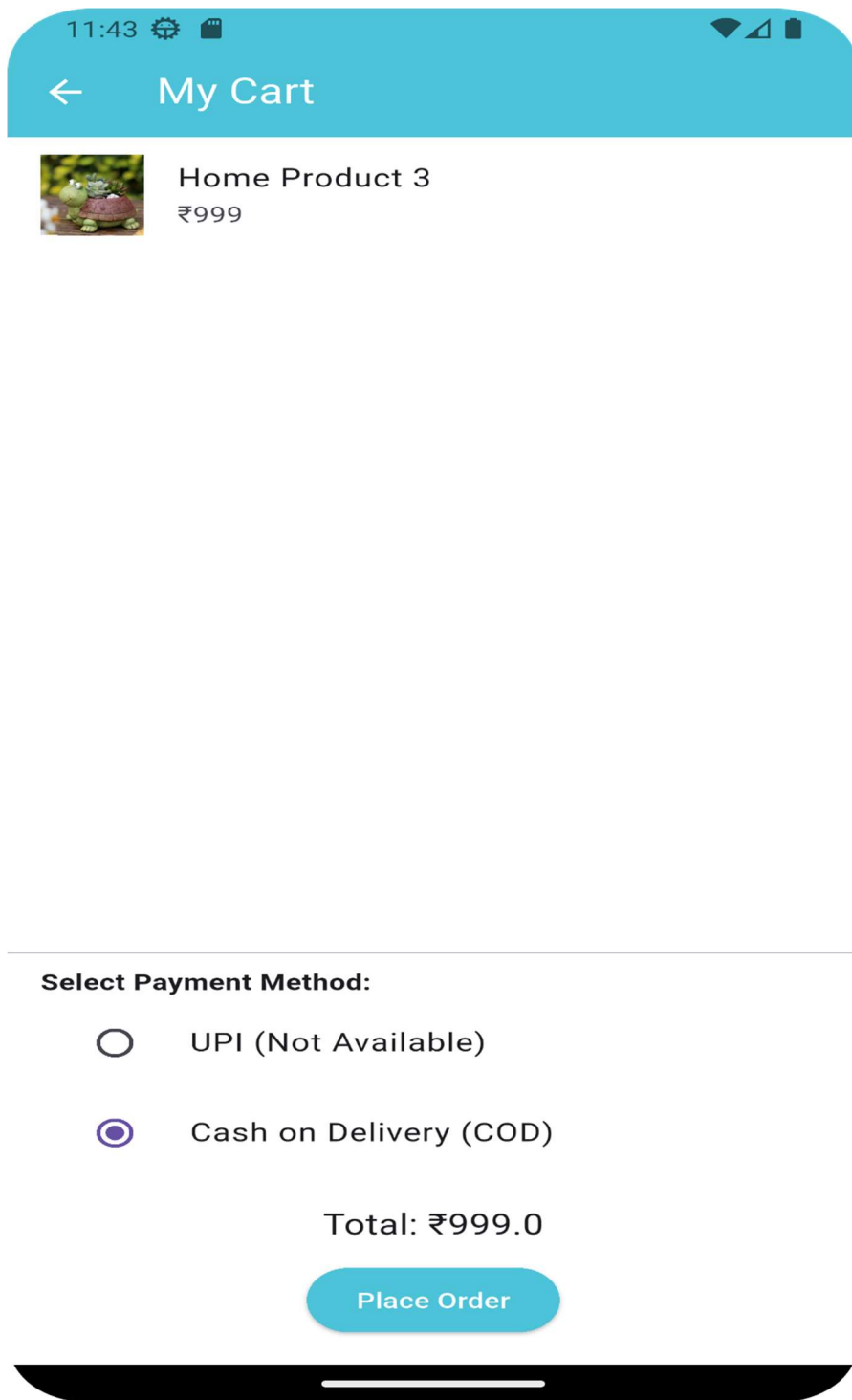
Gesture Detection

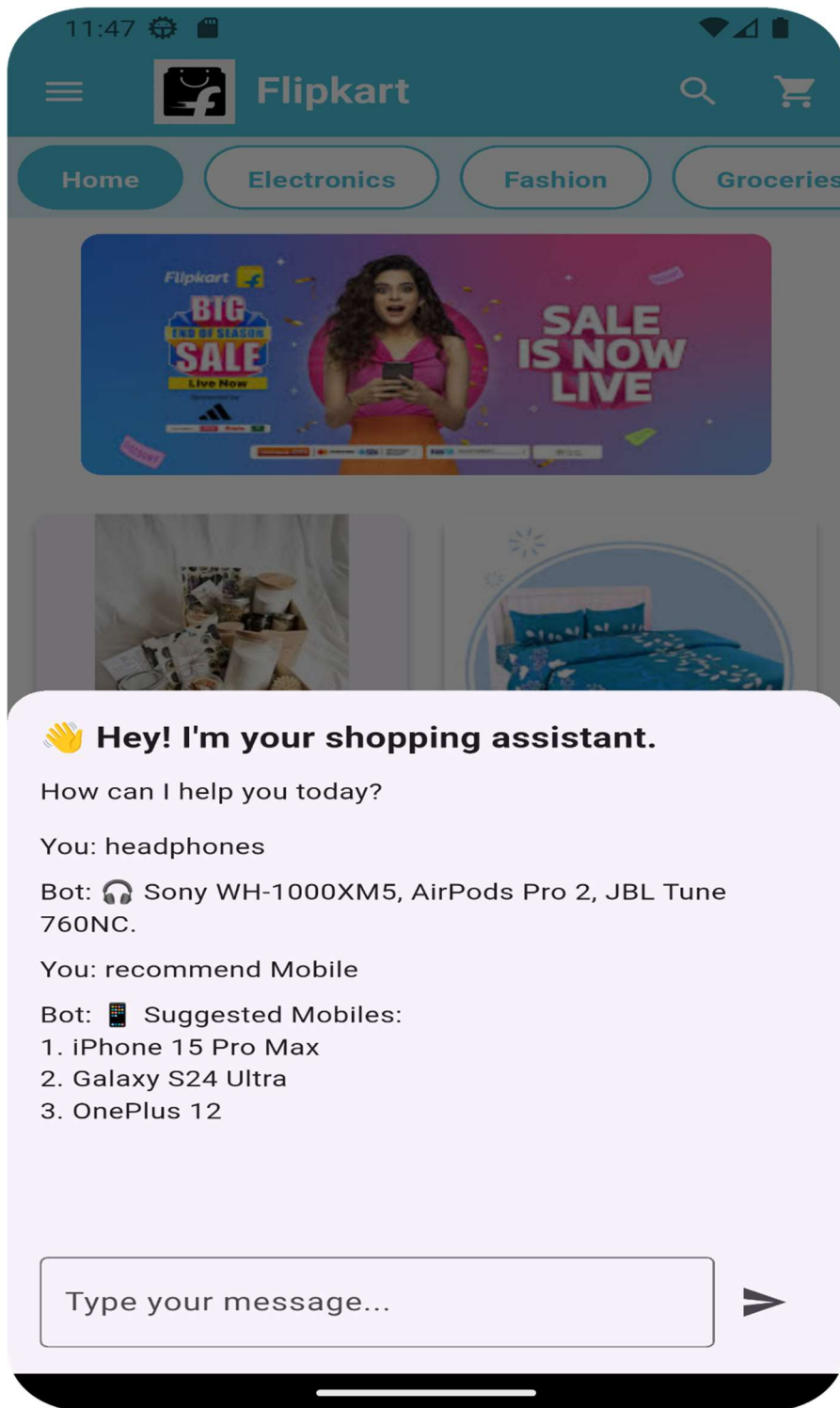
- Search bar

```
GestureDetector(
  onTap: () {
    setState(() {
      isExpanded = true;
    });
  },
  child: AnimatedContainer(
    // Container properties
  ),
);
```

The app opens with a splash screen, then moves to the welcome screen. Clicking "Sign Up" takes the user to the sign-up page, and after registering, they reach the home screen. Here, a "Shop Now" button leads to the cart, where users can review their items and billing details. Finally, clicking "Proceed to Checkout" takes them to a form to complete their purchase.

Screenshot:





11:46



Create Account

 Full Name

 Email

 Password

Sign Up

Already have an account? [Login](#)

localhost:58875

localhost:58875

Checkout

Full Name

Phone Number

Address

City

State

PIN Code

Payment Method
Card

Personalized Message (Optional)

Quantity
1

Place Order

MAD & PWA Lab

Journal

Experiment No.	06
Experiment Title.	To Connect Flutter UI with fireBase database
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

EXPERIMENT NO: 06

Name: Aditi Taksale

Class: D15A

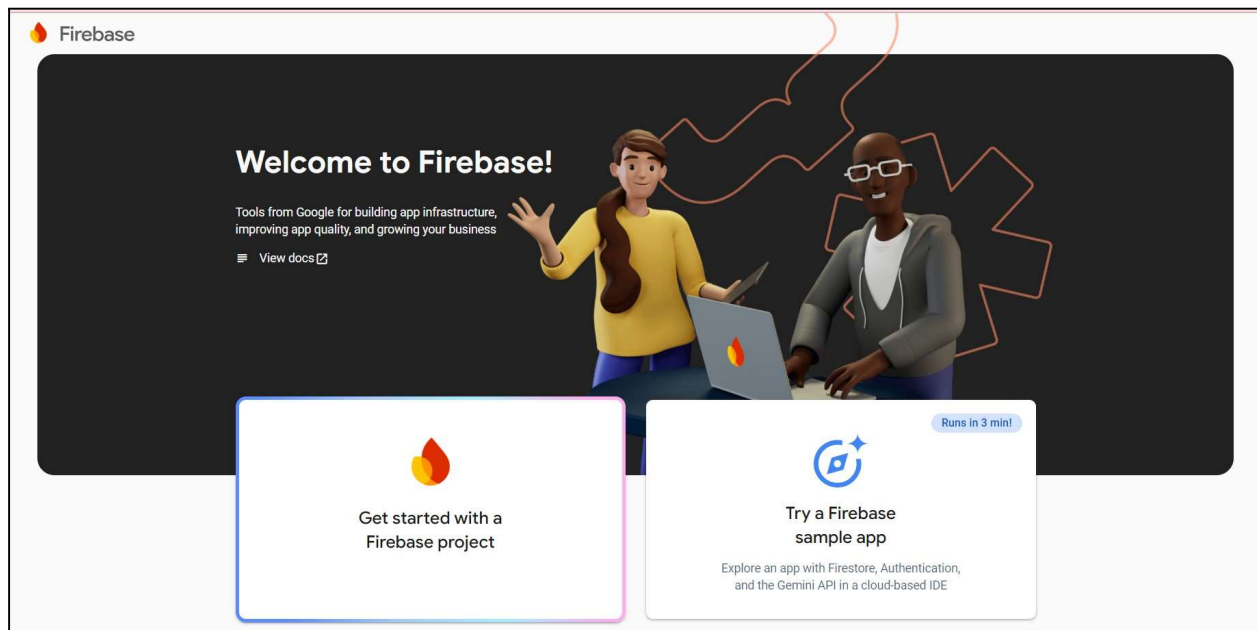
Roll No: 58

Aim: to connect flutter UI with Firebase

Steps:

Create a Firebase Project

- Go to Firebase Console.
- Click Create a project, enter name, and disable Google Analytics (optional).
- Click Continue to access the dashboard.



× Create a project

Let's start with a name for your project [?]

Project name
ETSYCLONE

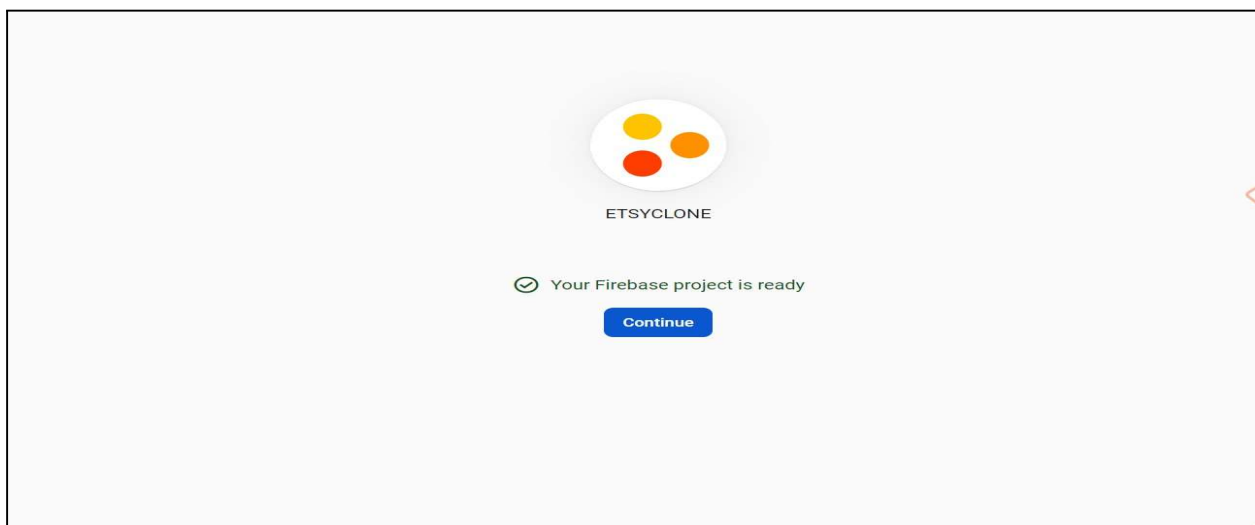
etsyclone-337ab Select parent resource

☒ I accept the [Firebase terms](#).

☒ I confirm that I will use Firebase exclusively for purposes relating to my trade, business, craft, or profession.

Already have a Google Cloud project?
[Add Firebase to Google Cloud project](#)

Continue



Register Your Flutter App

- Click **Add App** → Select **Android**.
- Enter package name (found in android/app/build.gradle).
- Click **Register App** and download google-services.json.
- Move google-services.json to android/app/ in your project.

×

1

②

```
com.example.etsy_clone
```

②

My Android App

②

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:(

Register app

2

3

×

Android package name: com.example.etsy_clone

2

Un

C+

+ [

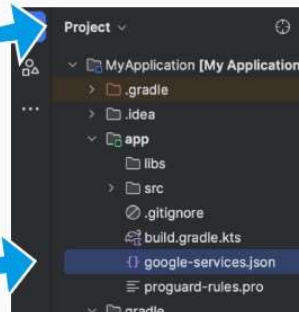
[Download google-services.json](#)

Switch to the **Project** view in Android Studio to see your project root directory.

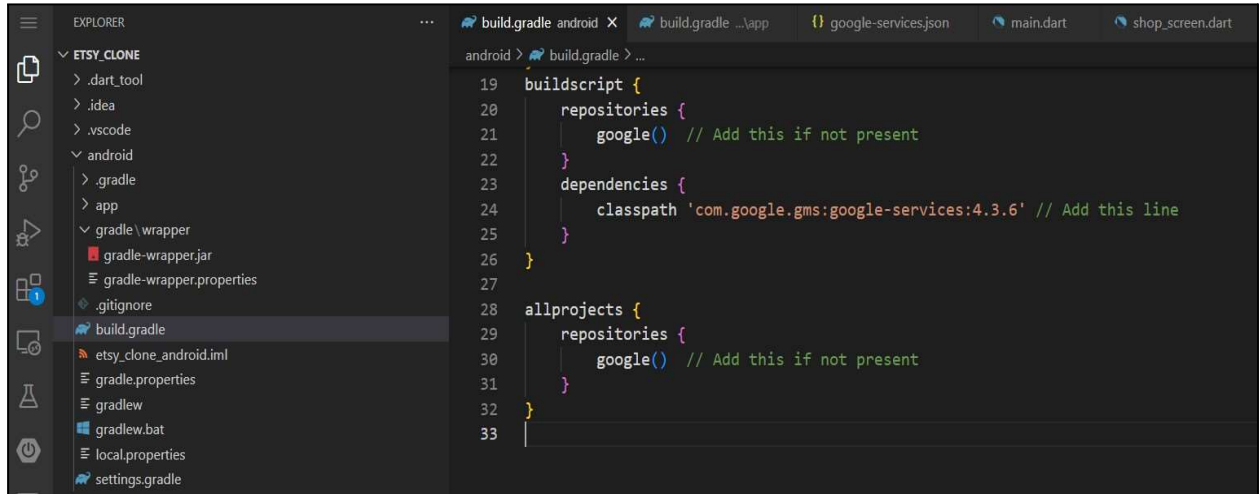
Move your downloaded `google-services.json` file into your module (app-level) root directory.



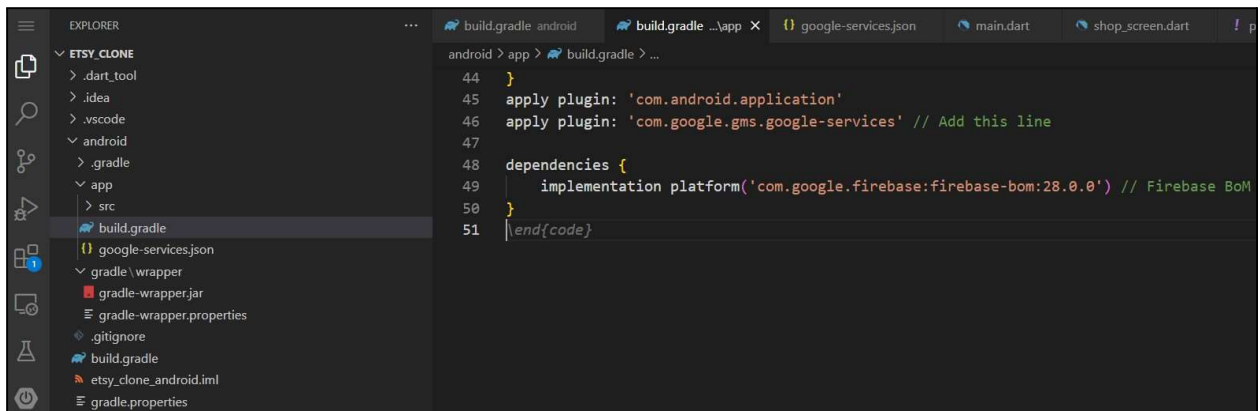
google-services.json



Next



- Configure Firebase in Flutter



3

Add Firebase SDK

Instructions for Gradle | [Unity](#) [C++](#)

★

Are you still using the `buildscript` syntax to manage plugins? Learn how to [add Firebase plugins](#) using that syntax.

1.

To make the `google-services.json` config values accessible to Firebase SDKs, you need the Google services Gradle plugin.

☐

Kotlin DSL (`build.gradle.kts`)

☒

Groovy (`build.gradle`)

Add the plugin as a dependency to your **project-level** `build.gradle` file:

Root-level (project-level) Gradle file (`<project>/build.gradle`):

```
plugins {  
    // ...  
  
    // Add the dependency for the Google services Gradle plugin  
    id 'com.google.gms.google-services' version '4.4.2' apply false  
}
```

2.

Then, in your **module (app-level)** `build.gradle` file, add both the `google-services` plugin and any Firebase SDKs that you want to use in your app:

Module (app-level) Gradle file (`<project>/<app-module>/build.gradle`):

```
plugins {  
    id 'com.android.application'  
    // Add the Google services Gradle plugin  
    id 'com.google.gms.google-services'
```

×

Add Firebase to your Android app

✓

Register app

Android package name: com.example.etsy_clone

🔗

Download and then add config file

🔗

Add Firebase SDK

4

Next steps

You're all set!

Make sure to check out the [documentation](#) to learn how to get started with each Firebase product that you want to use in your app.

You can also explore [sample Firebase apps](#).

Or, continue to the console to explore Firebase.

Previous

Continue to console

- Run flutterfire configure

```

PS C:\Documents\etsy_clone> flutterfire configure
>>
i Found 1 Firebase projects.
✓ Select a Firebase project to configure your Flutter application with · etsyclone-3920a (ETSYCLONE)
✓ Which platforms should your configuration support (use arrow keys & space to select)? · android, ios, macos, web, windows
i Firebase android app com.example.etsy_clone registered.
i Firebase ios app com.example.etsyClone is not registered on Firebase project etsyclone-3920a.
i Registered a new Firebase ios app on Firebase project etsyclone-3920a.
i Firebase macos app com.example.etsyClone registered.

```

- Initialize Firebase in main.dart

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  print("Firebase has been initialized successfully!");
  runApp(EtsyClone()); // Use EtsyClone instead of MyApp
}

```

```

PS C:\Documents\etsy_clone> flutter run
Connected devices:
Windows (desktop) • windows • windows-x64 • Microsoft Windows [Version 10.0.22631.4890]
Chrome (web) • chrome • web-javascript • Google Chrome 133.0.6943.127
Edge (web) • edge • web-javascript • Microsoft Edge 133.0.3065.69
[1]: Windows (windows)
[2]: Chrome (chrome)
[3]: Edge (edge)
Please choose one (or "q" to quit): 2
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome... 25.8s
This app is linked to the debug service: ws://127.0.0.1:54216/w20w2sbpgeg=/ws
Debug service listening on ws://127.0.0.1:54216/w20w2sbpgeg=/ws

To hot restart changes while running, press "r" or "R".
For a more detailed help message, press "h". To quit, press "q".

A Dart VM Service on Chrome is available at: http://127.0.0.1:54216/w20w2sbpgeg=
The Flutter DevTools debugger and profiler on Chrome is available at: http://127.0.0.1:9101?uri=http://127.0.0.1:54216/w20w2sbpgeg=
Firebase has been initialized successfully!

```

ETSYCLONE

Authentication

Users

Sign-in method

Templates

Usage

Settings

Extensions

Sign-in providers

Provider

Email/Password

Status

Enabled

Add new provider

ETSYCLONE

Authentication

Users

Sign-in method

Templates

Usage

Settings

Extensions

The following Authentication features will stop working when Firebase Dynamic Links shuts down on August 25, 2025: email link authentication for mobile apps, as well as Cordova OAuth support for web apps.

Search by email address, phone number, or user UID

Add user

Identifier	Providers	Created	Signed In	User UID
abc@gmail.com		Feb 22, 2025	Feb 22, 2025	gKqm2bvWb4R1AOtEBu82eN...
arya@gmail.com		Feb 22, 2025	Feb 22, 2025	2q9ba5cNfDh581dO6iB7oHLv...
priya@gmail.com		Feb 22, 2025	Feb 22, 2025	TbOPBKb5Sbxd5HEHFn5Qa...
shrushti@gmail.com		Feb 22, 2025	Feb 22, 2025	RsSdado9PQWxzAfLpyMx8fO...
kriti@gmail.com		Feb 22, 2025	Feb 22, 2025	hg7qOAG6qMOHLaJaiLAc3Cr...
shreyaaa@gmail.com		Feb 22, 2025	Feb 22, 2025	hwz1GrsdffY86yqm0l206kOz...
shreya647@gmail.com		Feb 22, 2025		6kRkdhpcjzej7JNAXecskM2gl...

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	58
Name	Aditi Talsale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

Experiment.7

Aim:- To write metadata of your E-commerce PWA in a Web app manifest file to enable the “add to home screen feature”.

Theory-

❖ What is a Progressive Web App (PWA)?

A **Progressive Web App (PWA)** is a type of web application that leverages modern web technologies to provide a user experience similar to a native mobile application. PWAs combine the benefits of both web and mobile apps, offering features such as offline access, push notifications, and installation on a user’s home screen.

Difference Between Regular Web App and PWA

A regular web app is a website designed to be accessible on mobile and desktop devices through a browser. However, a regular web app relies on the browser for functionality and does not support offline access or installation. In contrast, PWAs enhance the user experience by combining the best aspects of web and native apps. PWAs are installable, work offline, load quickly, and offer a seamless app-like experience.

❖ Advantages of PWA

1. **Faster Loading:** PWAs cache assets and serve them quickly, even under poor network conditions.
2. **Offline Support:** Content remains accessible even without an internet connection using service workers.
3. **App-like Experience:** PWAs run in a standalone window, offering a full-screen experience.
4. **Easy Installation:** Users can add the app to their home screen without needing to download it from an app store.
5. **Push Notifications:** Allows engagement with users even when the app is not active.
6. **Real-time Updates:** PWAs automatically update in the background, ensuring users always access the latest version.

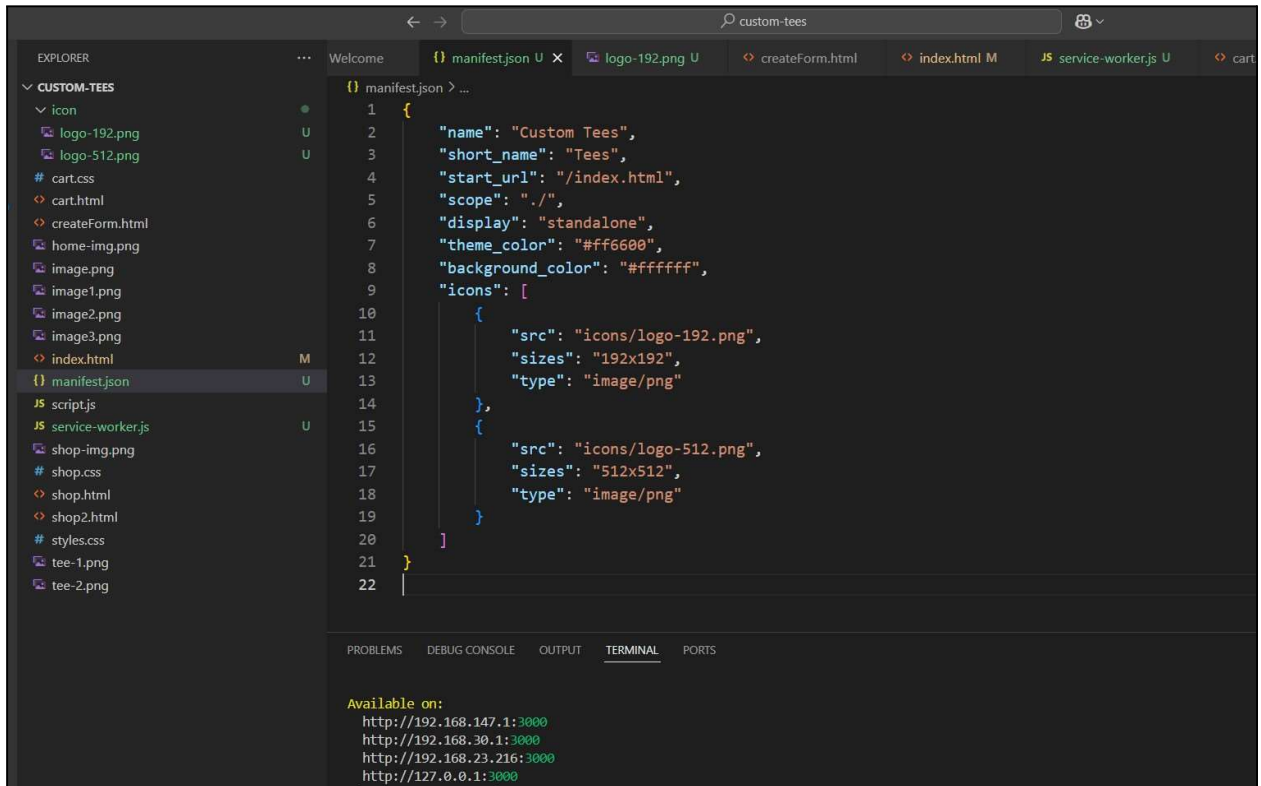
❖ What is a Web App Manifest File?

A **Web App Manifest** is a simple JSON file that defines metadata about the PWA. It includes information such as the app name, icons, start URL, theme color, background color, and display mode. The manifest file allows the app to be installed on the user's home screen and provides a consistent look and feel across different platforms.

How the "Add to Home Screen" Feature Works

The "Add to Home Screen" feature allows users to install the PWA on their device directly from the browser. When the user visits a PWA-compliant website, the browser prompts the user to install the app. Once installed, the app appears on the home screen like a native app and can be launched without opening the browser.

❖ Output:

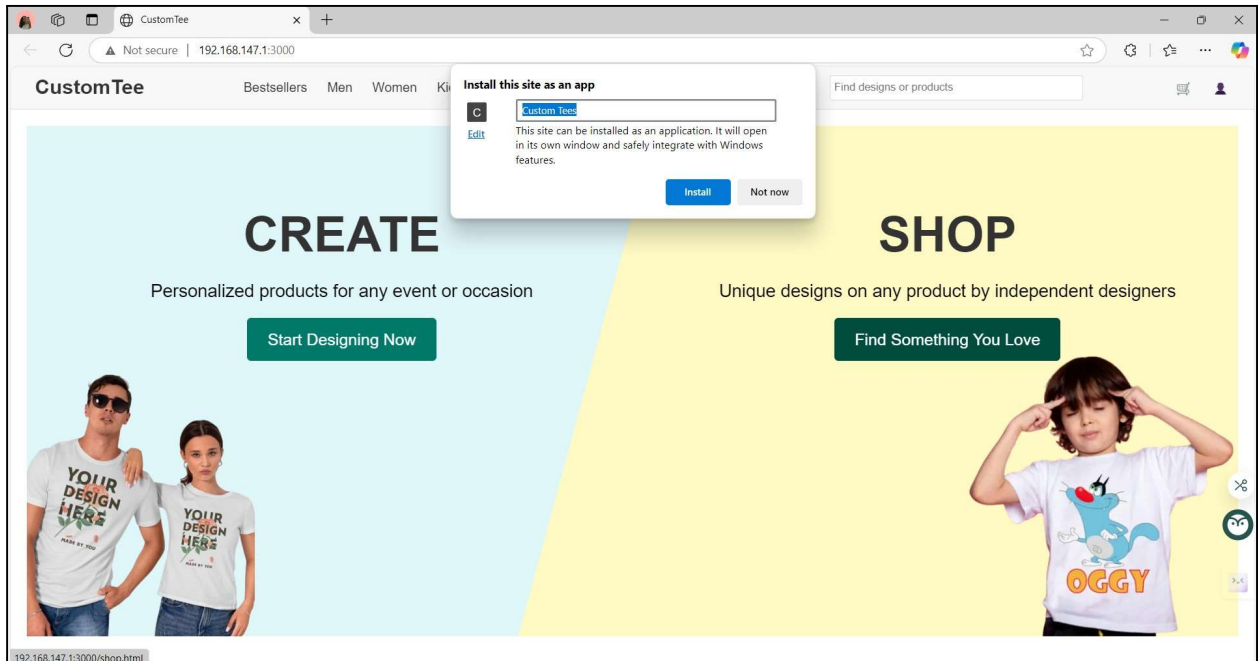
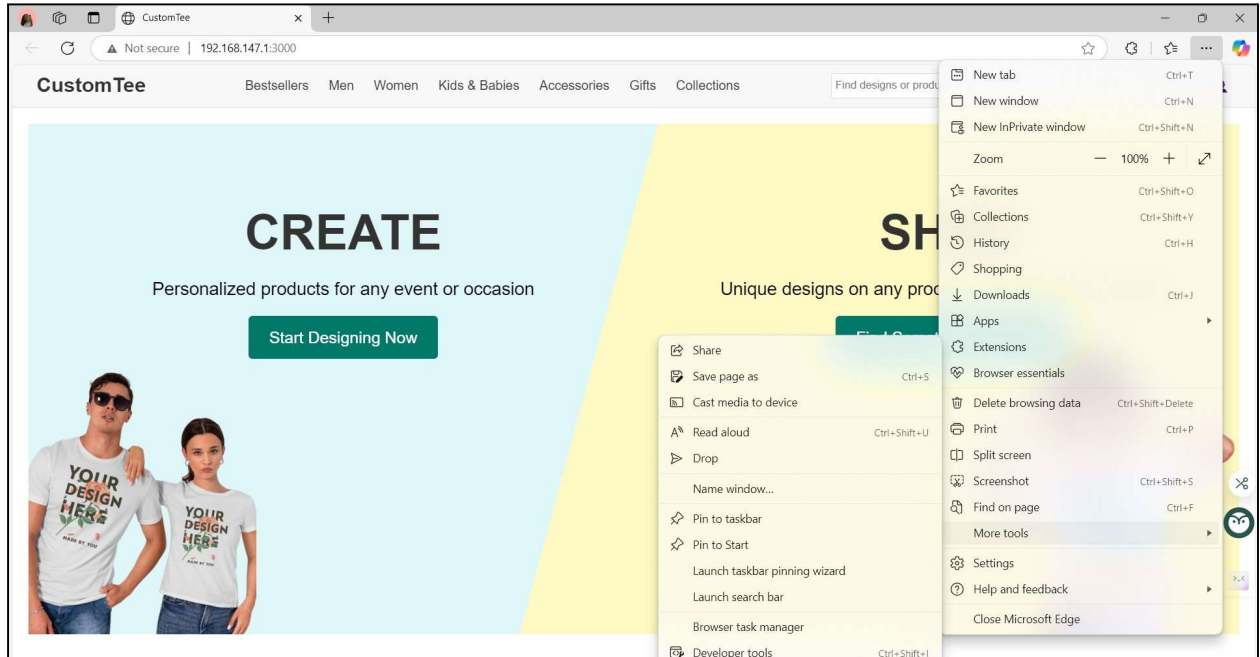


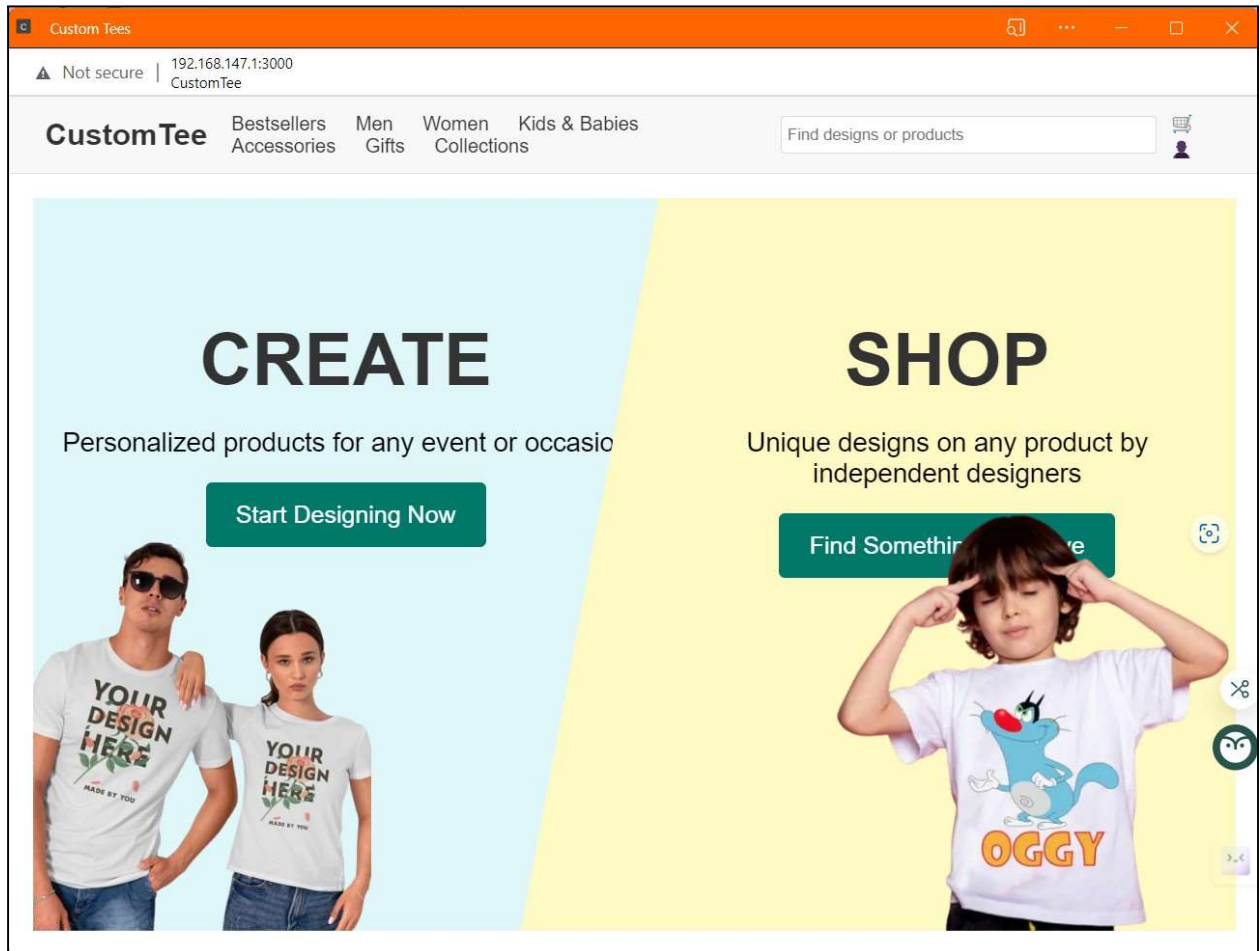
The screenshot shows a web browser window with a dark theme. The address bar displays "custom-tees". The browser's tab bar shows several open tabs: "manifest.json U", "logo-192.png U", "createForm.html", "index.html M", "JS service-worker.js U", and "cart". The main content area displays the JSON content of the "manifest.json" file. The file is named "manifest.json" and contains the following JSON structure:

```
{
  "name": "Custom Tees",
  "short_name": "Tees",
  "start_url": "/index.html",
  "scope": "/",
  "display": "standalone",
  "theme_color": "#ff6600",
  "background_color": "#ffffff",
  "icons": [
    {
      "src": "icons/logo-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/logo-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

At the bottom of the browser window, the "TERMINAL" tab is active, showing the following output:

```
Available on:
http://192.168.147.1:3000
http://192.168.30.1:3000
http://192.168.23.216:3000
http://127.0.0.1:3000
```





❖ Conclusion

Hence, we successfully created metadata for an E-commerce Progressive Web App (PWA) using a Web App Manifest file. The manifest file defines the app's name, icons, start URL, and display options, enabling the "Add to Home Screen" feature. Once installed, the PWA behaves like a native mobile application, providing a faster and more engaging user experience. PWAs offer benefits like offline access, push notifications, and real-time updates, making them a powerful alternative to traditional web and mobile apps.

MAD & PWA Lab

Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment.8

Aim:

To implement and demonstrate the working of Service Worker events like fetch, sync, and push in an E-commerce Progressive Web Application (PWA).

Theory:

Service Worker:

A Service Worker is a script that runs in the background of the browser, separate from the main thread. It acts as a programmable network proxy and is a core technology used in Progressive Web Applications (PWAs). Service workers enable features such as offline support, background data synchronization, and push notifications, thereby enhancing the reliability and functionality of web applications.

Key Characteristics of Service Workers:

- **Background Operation:** They continue to work even when the application is not active.
- **HTTPS Requirement:** They only run on secure origins to ensure user safety.
- **Event-Driven Architecture:** All operations are event-based and rely heavily on Promises for asynchronous handling.
- **No DOM Access:** Service workers operate independently and interact with the web page using the `postMessage` API.

Fetch Event:

The `fetch` event is triggered every time the web application makes a network request. The service worker can intercept these requests and respond with cached content or fetch fresh data from the network.

Common Strategies:

1. Cache First:

- Checks the cache before making a network request.
- Returns cached response if available; otherwise fetches from the network.

2. Network First:

- Tries to fetch the resource from the network.
- Falls back to the cached version if the network is unavailable.

Sync Event (Background Synchronization):

The **sync** event allows the service worker to defer actions until the user is online. It is useful for tasks such as submitting data that was generated while the user was offline.

Typical Workflow:

1. The user initiates an action (e.g., form submission) while offline.
2. Data is saved locally, usually in IndexedDB.
3. A sync event is registered using `registration.sync.register('sync-data')`.
4. Once the connection is restored, the service worker is activated and performs the pending task.

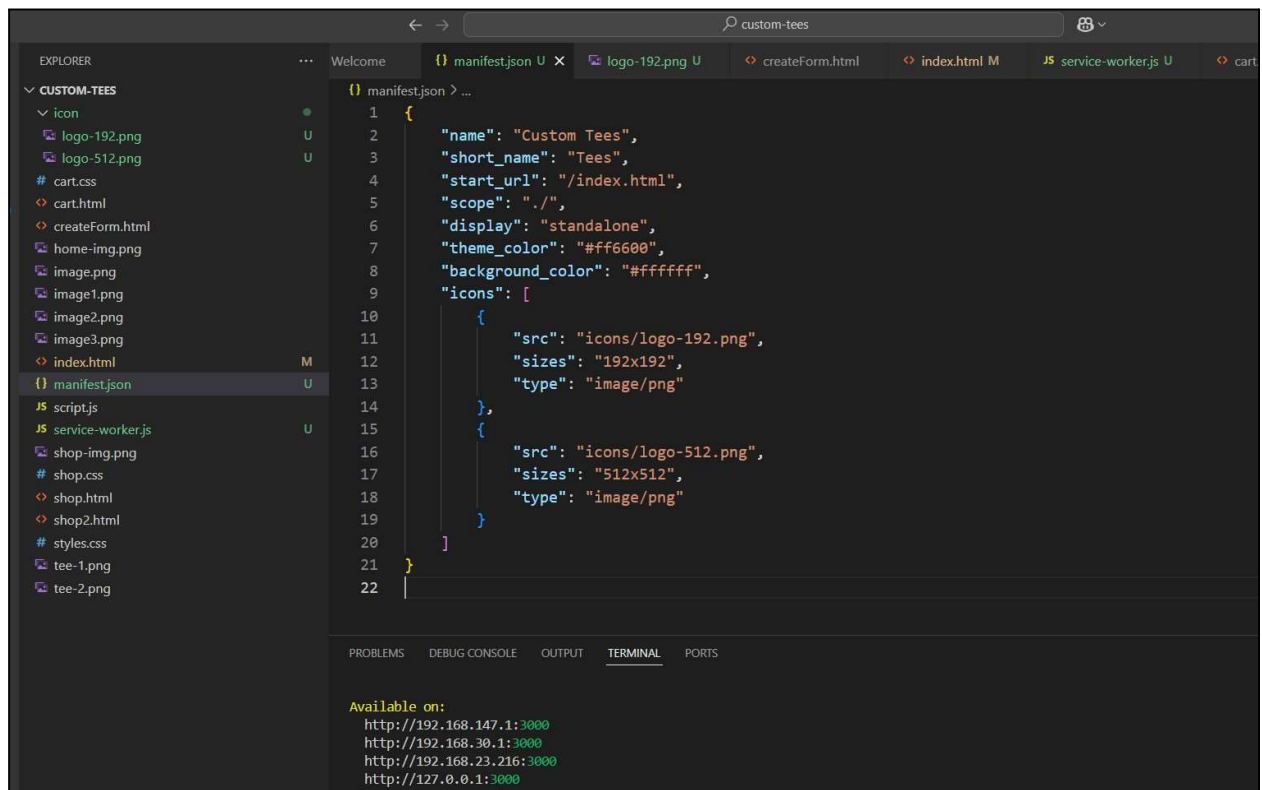
Push Event:

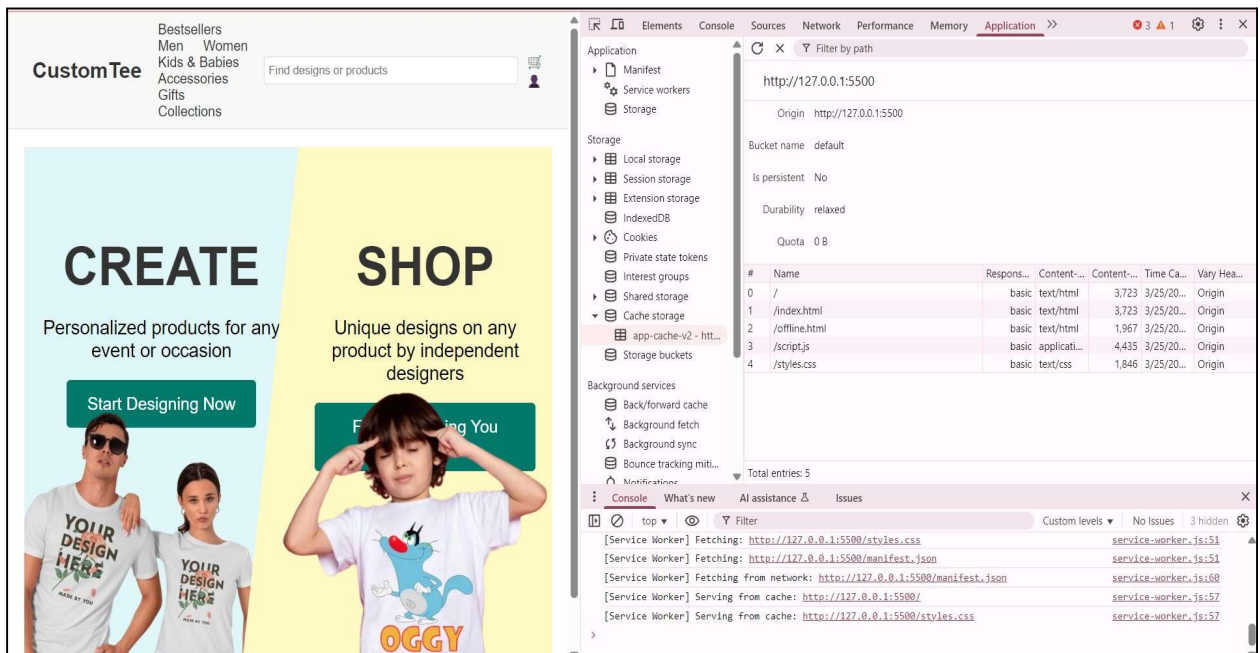
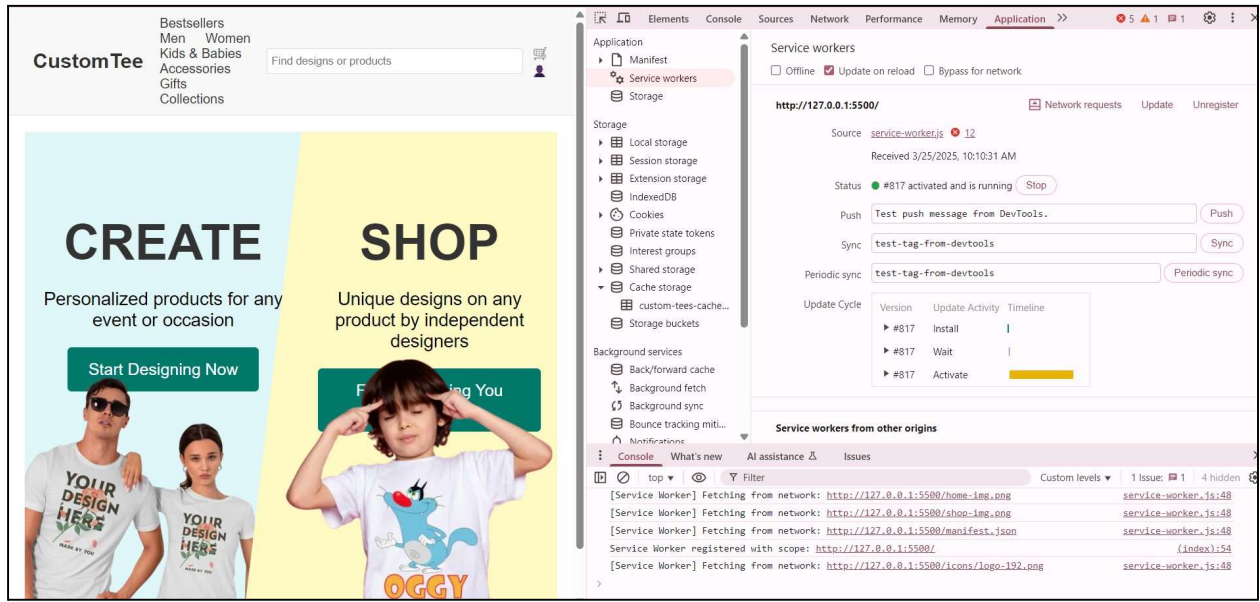
The **push** event is fired when the browser receives a push message from a server. This allows the service worker to display a notification, even when the application is not currently in use.

Workflow:

1. The client subscribes to a push service using the Push API.
2. The server sends a push message to the subscribed client.
3. The service worker handles the **push** event and displays a notification using the Notification API.

Output:





Conclusion:

Service workers are an essential part of modern web applications, especially PWAs. They provide the ability to work offline, delay background tasks until a network is available, and engage users with real-time notifications. The **fetch**, **sync**, and **push** events enable powerful features that enhance user experience, reliability, and interactivity of web apps, even in challenging network conditions.

MAD & PWA Lab

Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment.9

Aim: To implement and demonstrate the working of Service Worker events like fetch, sync, and push in an E-commerce Progressive Web Application (PWA).

Theory:

Service Worker: A Service Worker is a script that runs in the background of a browser independently of a web page. It acts as a programmable network proxy on the client side and is designed to enhance the capabilities of web apps by enabling features like offline support, background sync, and push notifications.

Key features of a Service Worker:

- Service workers intercept and handle network requests, allowing for smart caching and background content fetching.
- They only work over HTTPS for security reasons.
- Service workers become idle when not in use and restart when needed.
- They are heavily based on Promises and asynchronous event handling.

Fetch Event: The fetch event enables the service worker to intercept network requests made by the application. There are typically two strategies used:

1. Cache First:

- Check if the request exists in the cache.
- If available, return the cached response.
- Otherwise, fetch from the network.

2. Network First:

- Attempt to fetch from the network first.
- If successful, update the cache with the new response.
- If the network request fails, return the cached response if available.

Sync Event (Background Sync): Background Sync allows tasks like data submission to be deferred until the device has a stable internet connection. It is

useful in scenarios like email sending or form submission in poor network conditions.

Workflow:

1. User initiates an action (e.g., form submission) when offline.
2. Data is stored locally (e.g., in IndexedDB).
3. The service worker registers a sync event.
4. When online, the service worker performs the action (e.g., sending data to the server).

Push Event: The push event is triggered when a push notification is sent from the server to the client. Service workers can handle these notifications and display them even if the web app is not currently open.

Workflow:

1. The server sends a push message to the service worker.
2. The service worker receives the push event and displays a notification.
3. The user can interact with the notification, which can trigger further actions.

Code :

Service Worker.js

```
const CACHE_NAME = 'custom-tee-v1';
const FILES_TO_CACHE = [
  '/',
  '/index.html',
  '/cart.html',
  '/createForm.html',
  '/shop.html',
  '/shop2.html',
  '/offline.html',
  '/styles.css',
  '/cart.css',
  '/shop.css',
  '/script.js',
```

```
    '/manifest.json',  
    '/tee-1.png',  
    '/tee-2.png',  
    '/home-img.png',  
    '/shop-img.png',  
    '/image.png',  
    '/image1.png',  
    '/image2.png',  
    '/image3.png'  
  ];  
};
```

// Install Service Worker

```
self.addEventListener('install', (event) => {  
  console.log('[Service Worker] Install');  
  event.waitUntil(  
    caches.open(CACHE_NAME).then((cache) => {  
      console.log('[Service Worker] Caching files');  
      return cache.addAll(FILE_TO_CACHE);  
    })  
  );  
  self.skipWaiting();  
});
```

// Activate Service Worker

```
self.addEventListener('activate', (event) => {  
  console.log('[Service Worker] Activate');  
  event.waitUntil(  
    caches.keys().then((keyList) =>  
      Promise.all(  
        keyList.map((key) => {  
          if (key !== CACHE_NAME) {  
            console.log('[Service Worker] Removing old cache', key);  
            return caches.delete(key);  
          }  
        })  
      )  
    )  
  );  
});
```

```
);  
self.clients.claim();  
});
```

```
// Fetch
```

```
self.addEventListener('fetch', (event) => {  
  console.log('[Service Worker] Fetch', event.request.url);  
  event.respondWith(  
    fetch(event.request)  
      .then((response) => {  
        return caches.open(CACHE_NAME).then((cache) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
      })  
      .catch(() => {  
        return caches.match(event.request).then((response) => {  
          return response || caches.match('/offline.html');  
        });  
      })  
  );  
});
```

```
// SYNC
```

```
self.addEventListener('sync', function(event) {  
  console.log('🔄 Sync event triggered:',  
    event.tag); if (event.tag === 'custom-tees-sync') {  
    event.waitUntil(  
      Promise.resolve().then(() => {  
        console.log('✅ Sync  
successful');  
      })  
    );  
  }  
});
```

Bestsellers

Men

Women

Kids & Babies

Accessories

Gifts

Collections

CustomTee

Find designs or products

CREATE

SHOP

Personalized products for any event or occasion

Unique designs on any product by independent designers

Start Designing Now

Something



Application

Manifest

Service workers

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking mitl...

Notifications

Payment handler

http://127.0.0.1:5500/

Source: service-worker.js

Received 4/7/2025, 2:34:17 PM

Status: #900 activated and is running

Stop

Clients: http://127.0.0.1:5500/

Push: { "method": "pushMessage", "message": "Hello from P" }

Push

Sync: hellosync

Sync

Periodic sync: test-tag-from-devtools

Periodic sync

Update Cycle

Version	Update Activity	Timeline
#900	Install	
#900	Wait	
#900	Activate	

Service workers from other origins

See all registrations

Console

AI assistance

Issues

top

Filter

[Service Worker] Fetching: http://127.0.0.1:5500/manifest.json service-worker.js:65

[Service Worker] Serving from cache: http://127.0.0.1:5500/styles.css service-worker.js:69

[Service Worker] Serving from cache: http://127.0.0.1:5500/ service-worker.js:69

[Service Worker] Serving from cache: http://127.0.0.1:5500/manifest.json service-worker.js:69

Notification displayed!

Bestsellers

Men

Women

Kids & Babies

Accessories

Gifts

Collections

CustomTee

Find designs or products

CREATE

SHOP

Personalized products for any event or occasion

Unique designs on any product by independent designers

Start Designing Now

Something



Application

Manifest

Service workers

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking mitl...

Notifications

Payment handler

Periodic background ...

Speculative loads

#

Timestamp

Event

Origin

Storage Key

Service ...

Instance ID

1 2025-04-07 15:44... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

2 2025-04-07 15:45... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

3 2025-04-07 15:45... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

4 2025-04-07 16:16... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

5 2025-04-07 16:16... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

6 2025-04-07 16:17... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

7 2025-04-07 16:17... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

8 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

9 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:31... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:32... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:32... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:36... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:36... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

No event selected

Select an event to view its metadata

Console

AI assistance

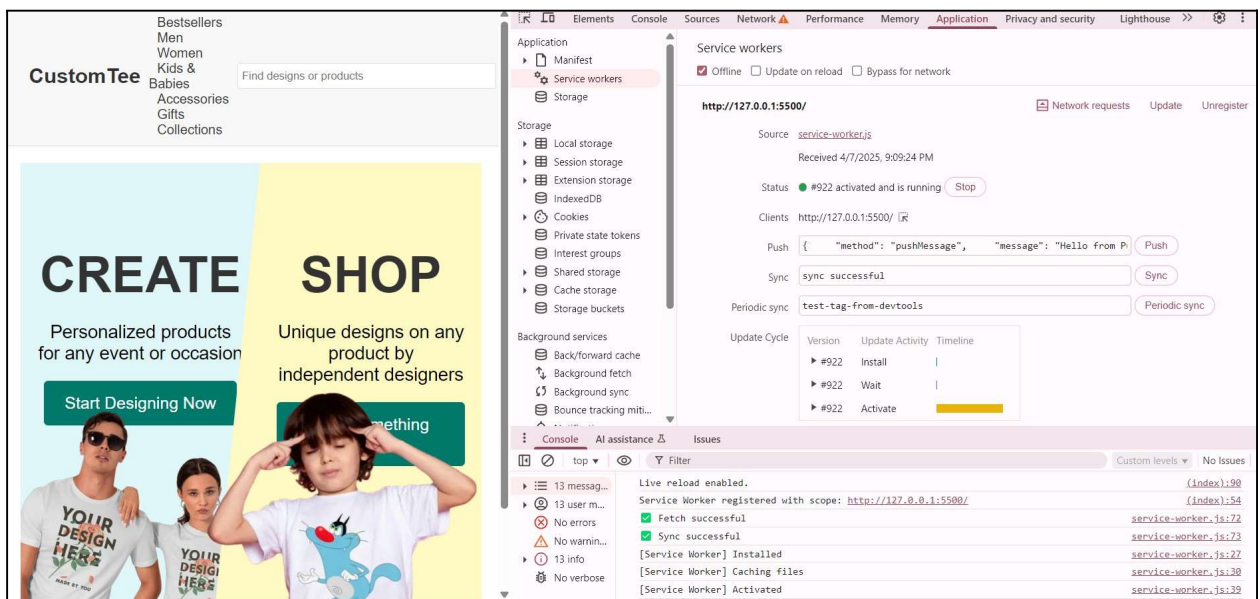
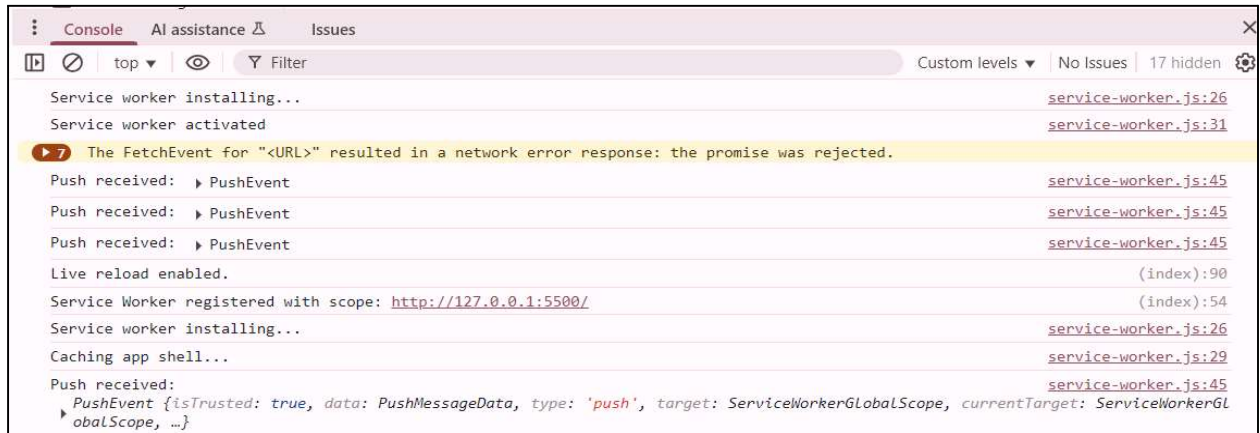
Issues

top

Filter

Push received: service-worker.js:45

PushEvent {isTrusted: true, data: PushMessageData, type: 'push', target: ServiceWorkerGlobalScope, currentTarget: ServiceWorkerGlobalScope, ...}



Conclusion: Service workers significantly enhance the functionality and reliability of web applications. With events like fetch, sync, and push, developers can provide offline capabilities, delay tasks until connectivity is restored, and engage users with timely notifications. These features make PWAs more robust and user-friendly, especially in low or intermittent network environments.

MAD & PWA Lab

Journal

Experiment No.	10
Experiment Title.	To study and implement deployment of Ecommerce PWA to GitHub Pages.
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Experiment No. 10

Aim:

To study and implement deployment of Ecommerce PWA to GitHub Pages.

Theory:

GitHub Pages

Public web pages are freely hosted and easily published. Public webpages hosted directly from your GitHub repository. Just edit, push, and your changes are live.

GitHub Pages provides the following key features:

1. Blogging with Jekyll
2. Custom URL
3. Automatic Page Generator

Reasons for favoring this over Firebase:

1. Free to use
2. Right out of github
3. Quick to set up

GitHub Pages is used by Lyft, CircleCI, and HubSpot.

GitHub Pages is listed in 775 company stacks and 4401 developer stacks.

Pros

1. Very familiar interface if you are already using GitHub for your projects.
2. Easy to set up. Just push your static website to the gh-pages branch and your website is ready.
3. Supports Jekyll out of the box.
4. Supports custom domains. Just add a file called CNAME to the root of your site, add an A record in the site's DNS configuration, and you are done.

Cons

1. The code of your website will be public, unless you pay for a private repository.
2. Currently, there is no support for HTTPS for custom domains. It's

probably coming soon though.

3. Although Jekyll is supported, plug-in support is rather spotty.

Firebase

The Realtime App Platform. Firebase is a cloud service designed to power real-time, collaborative applications. Simply add the Firebase library to your application to gain access to a shared data structure; any changes you make to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.

Some of the features offered by Firebase are:

1. Add the Firebase library to your app and get access to a shared data structure. Any changes made to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.
2. Firebase apps can be written entirely with client-side code, update in real-time out-of-the-box, interoperate well with existing services, scale automatically, and provide strong data security.
3. Data Accessibility- Data is stored as JSON in Firebase. Every piece of data has its own URL which can be used in Firebase's client libraries and as a REST endpoint. These URLs can also be entered into a browser to view the data and watch it update in real-time.

Reasons for favoring over GitHub Pages:

1. Realtime backend made easy
2. Fast and responsive

Instacart, 9GAG, and Twitch are some of the popular companies that use Firebase

Firebase has a broader approval, being mentioned in 1215 company stacks & 4651 developers stacks

Pros

1. Hosted by Google. Enough said.

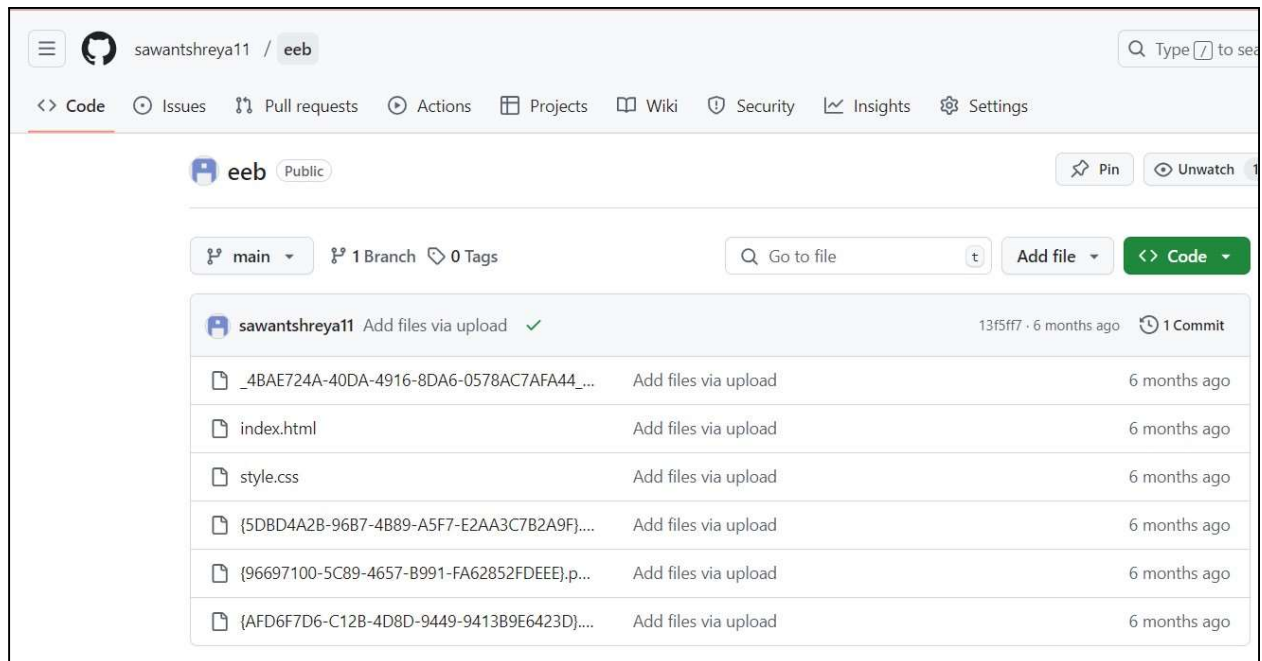
2. Authentication, Cloud Messaging, and a whole lot of other handy services will be available to you.
3. A real-time database will be available to you, which can store 1 GB of data.
4. You'll also have access to a blob store, which can store another 1 GB of data.
5. Support for HTTPS. A free certificate will be provisioned for your custom domain within 24 hours.

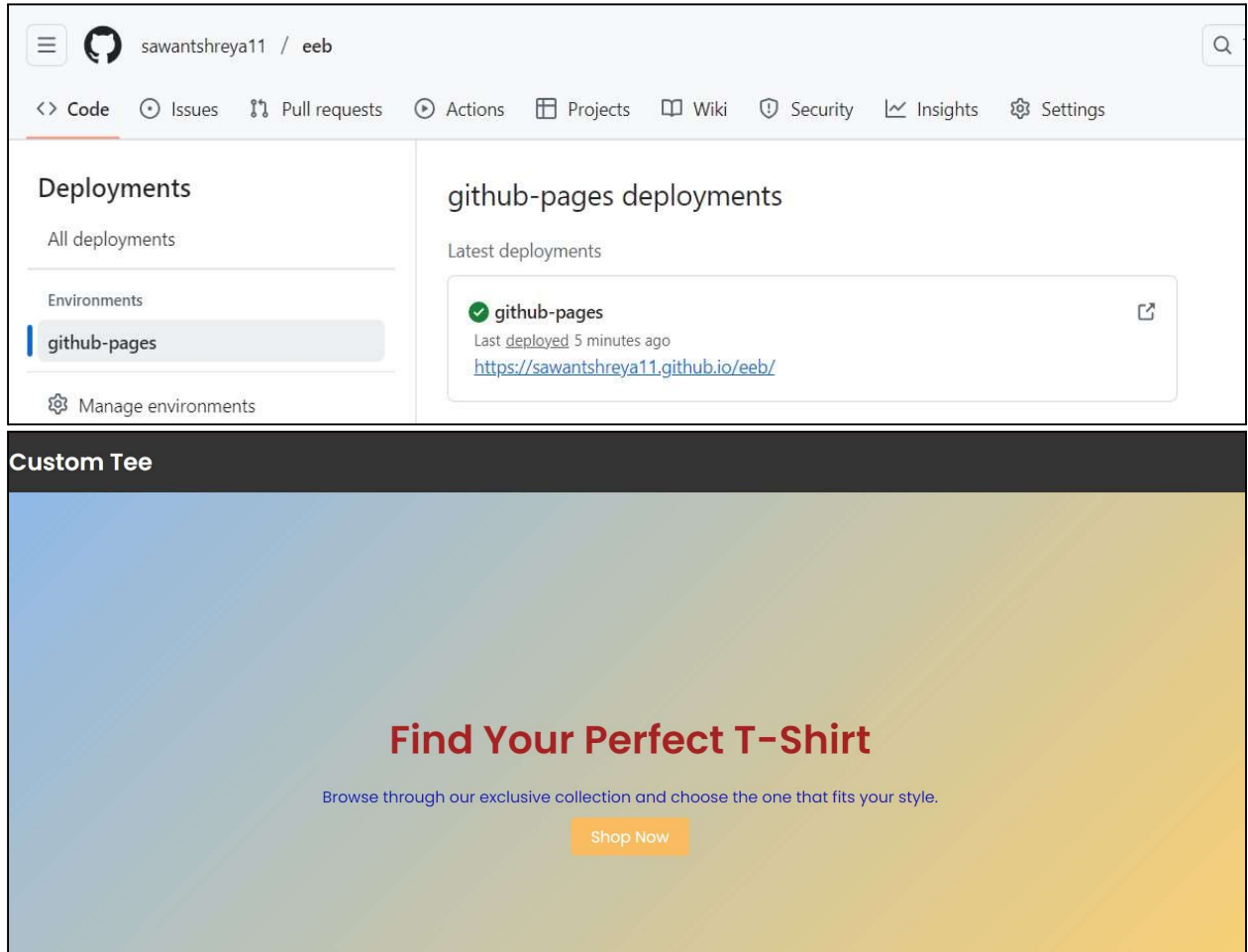
Cons

1. Only 10 GB of data transfer is allowed per month. But this is not really a big problem, if you use a CDN or AMP.
2. Command-line interface only.
3. No in-built support for any static site generator.

Link to our GitHub repository: <https://github.com/sawantshreya11/eeb>

Github Screenshot:





Deployed Link:<https://sawantshreya11.github.io/eeb/>

Conclusion:

In this experiment, we successfully deployed an Ecommerce PWA using GitHub Pages. The process was quick, free, and ideal for hosting static web applications. We also compared GitHub Pages with Firebase and found each has its own strengths depending on project requirements. The deployed app is now live and accessible online.

MAD & PWA Lab

Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	58
Name	Aditi Taksale
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

Experiment . 11

Aim:

To analyze the functionality and performance of a Progressive Web App (PWA) using the Google Lighthouse PWA Analysis Tool.

Theory:

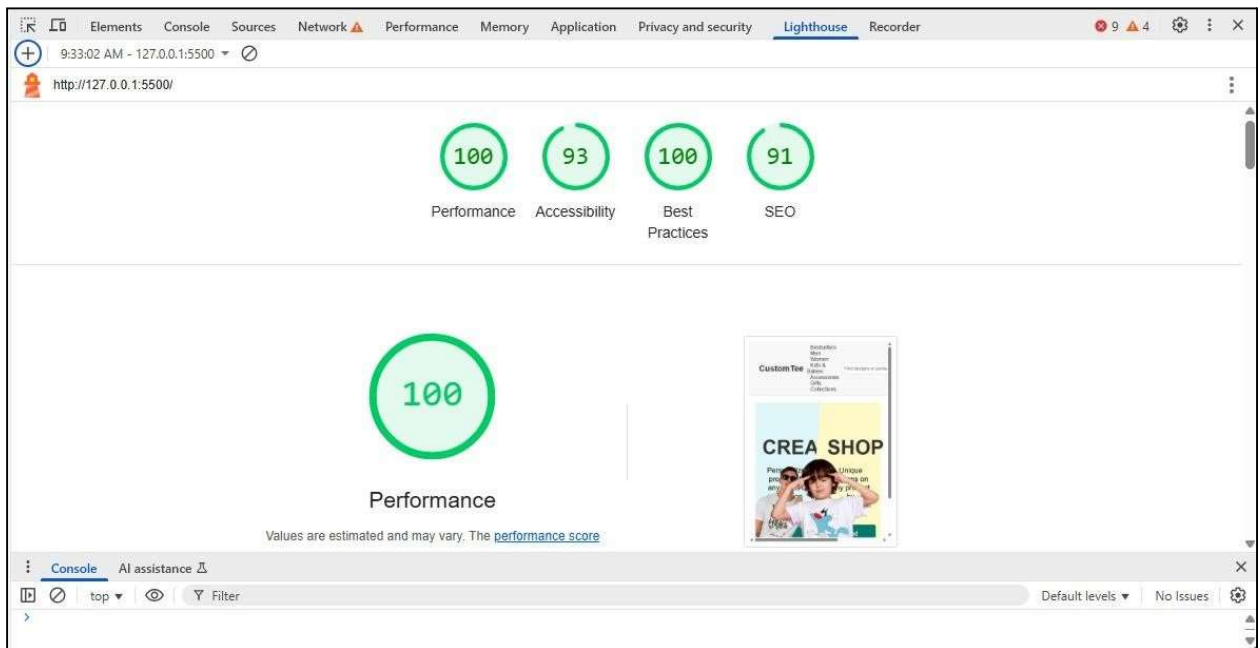
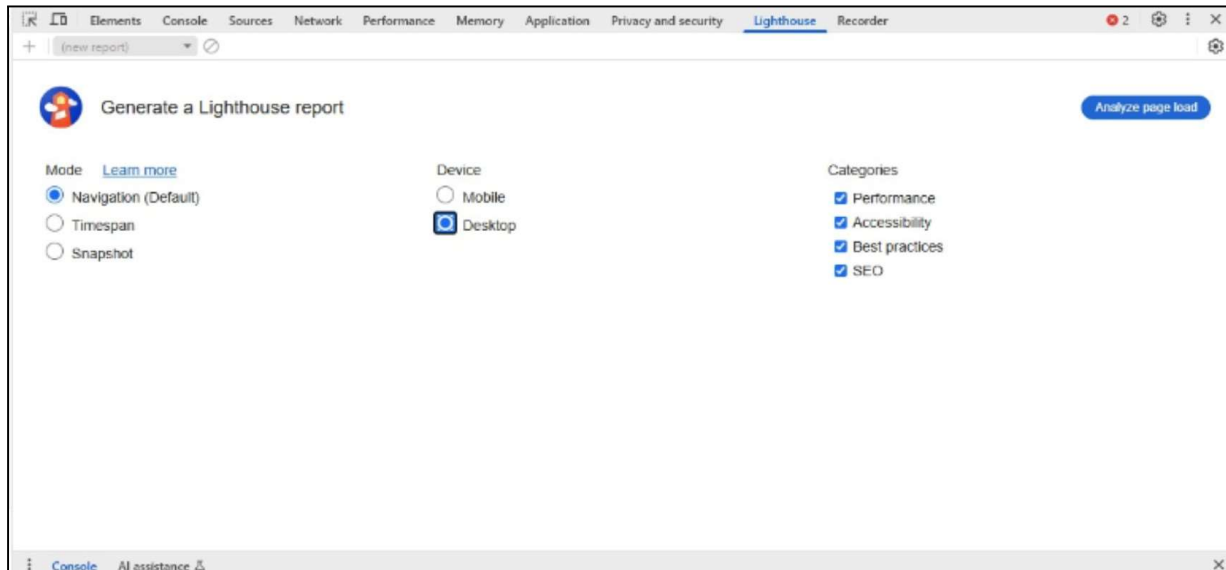
Google Lighthouse is an open-source, automated auditing tool developed by Google that assesses the quality of web pages. It performs audits across several categories including Performance, Accessibility, Best Practices, SEO, and Progressive Web App (PWA) compliance.

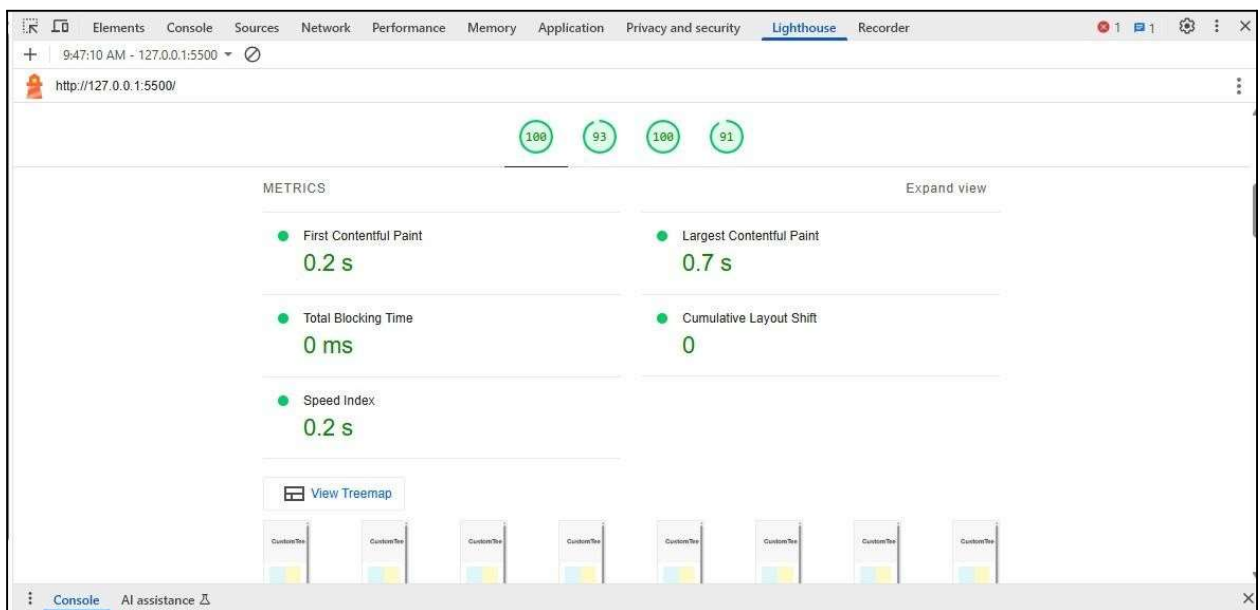
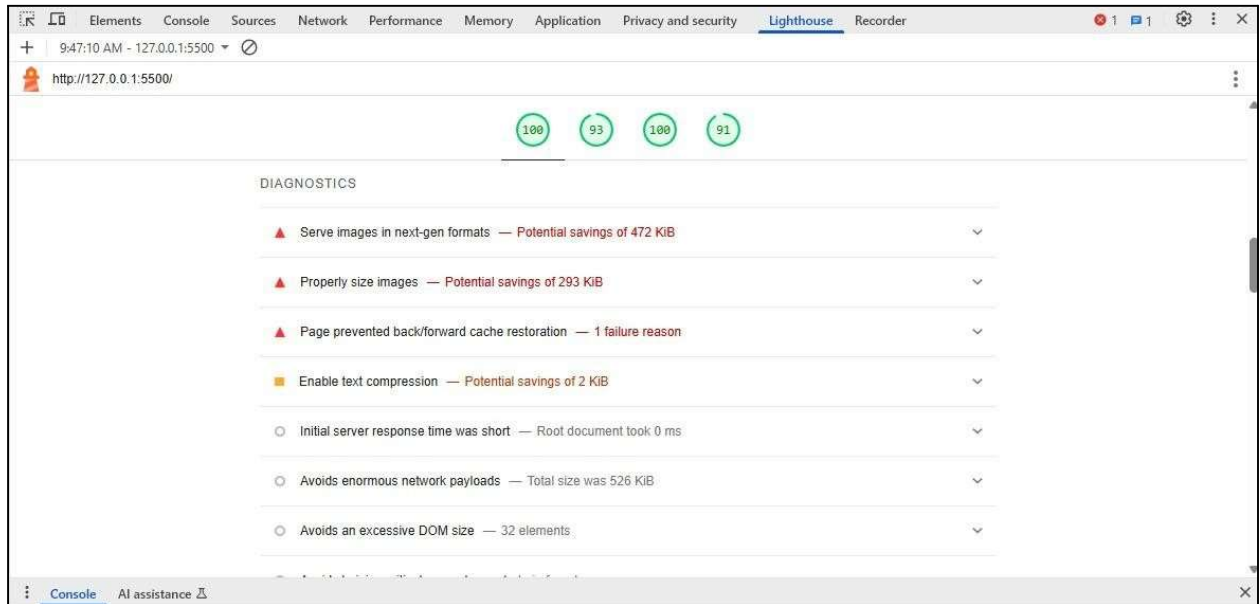
For PWAs, Lighthouse ensures that the application follows modern web standards such as:

- Presence and configuration of a **Service Worker** for offline capabilities
- A well-defined **Web App Manifest** with appropriate metadata (icons, theme color, display mode)
- **Responsive design** and proper **viewport configuration** for mobile compatibility
- **Installability** on user devices
- Offline functionality and adherence to **Google's baseline PWA checklist**

Additionally, Lighthouse scores the site on performance (loading speed and responsiveness), accessibility (support for screen readers and assistive tools), and web development best practices (secure connections, modern syntax, deprecated feature avoidance, etc.).

Output:





Conclusion:

Thus, we successfully used the Google Lighthouse PWA Analysis Tool to audit and verify the Progressive Web App functionality of our project. The report provided insights into areas of improvement, ensuring that our PWA meets modern standards for performance, accessibility, and installability across devices.

